Operationalize machine learning models (MLOps)

Experiment with Azure Machine Learning

Introduction

When you're asked to build a machine learning model, you rarely know up front which algorithm or preprocessing steps will give you the best results. Finding the right combination takes experimentation.

Azure Machine Learning gives you two ways to experiment efficiently. **Automated machine learning (AutoML)** searches through algorithms and preprocessing configurations automatically, running multiple training jobs in parallel. **Jupyter notebooks** let you write and iterate on your own training code, while **MLflow** tracks every run so you can compare results.

Preprocess data and configure featurization

Before you can run an automated machine learning (AutoML) experiment, you need to prepare your data. When you want to train a machine learning model, you only need to provide the training data.

After you collected the data, you need to create a **data asset** in Azure Machine Learning. In order for AutoML to understand how to read the data, you need to create a **MLTable** data asset that includes the schema of the data.

You can create a MLTable data asset when your data is stored in a folder together with a MLTable file. After you created the data asset, you can specify it as input with the following code:

```
from azure.ai.ml.constants import AssetTypes
from azure.ai.ml import Input

my_training_data_input = Input(type=AssetTypes.MLTABLE, path="azureml:input-data-automl:1")
```

> **Tip**: Learn more about how to create a MLTable data asset in Azure Machine Learning.

Once you created the data asset, you can configure the AutoML experiment. Before AutoML trains a classification model, preprocessing transformations can be applied to your data.

Understand scaling and normalization

AutoML applies scaling and normalization to numeric data automatically, helping prevent any large-scale features from dominating training. During an AutoML experiment, multiple scaling or normalization techniques are applied.

Configure optional featurization

You can choose to have AutoML apply preprocessing transformations, such as:

- Missing value imputation to eliminate nulls in the training dataset.
- Categorical encoding to convert categorical features to numeric indicators.
- Dropping high-cardinality features, such as record IDs.
- Feature engineering (for example, deriving individual date parts from DateTime features)

By default, AutoML performs featurization on your data. You can disable it if you don't want the data to be transformed.

If you do want to make use of the integrated featurization function, you can customize it. For example, you can specify which imputation method should be used for a specific feature.

After an AutoML experiment is completed, you can review which scaling and normalization methods were applied. You get notified if AutoML detected any issues with the data, like whether there are missing values or class imbalance.

Run an automated machine learning experiment

To run an automated machine learning (AutoML) experiment, you can configure and submit the job with the Python SDK.

The algorithms AutoML uses depends on the task you specify. For example, when you want to train a classification model, AutoML chooses from a list of classification algorithms:

- Logistic Regression
- Light Gradient Boosting Machine (GBM)
- Decision Tree
- Random Forest
- Naive Bayes
- Linear Support Vector Machine (SVM)
- XGBoost
- And others...

> **Tip**: For a full list of supported algorithms, explore the overview of supported algorithms.

Restrict algorithm selection

By default, AutoML randomly selects from the full range of algorithms for the specified task. You can choose to block individual algorithms from being selected; which can be useful if you know that your data isn't suited to a particular type of algorithm. You can block certain algorithms if you have to comply with a policy that restricts the type of machine learning algorithms you can use in your organization.

Configure an AutoML experiment

When you use the Python SDK (v2) to configure an AutoML experiment or job, you configure the experiment using the `automl` class. For classification, you use the `automl.classification` function as shown in the following example:

```
from azure.ai.ml import automl
```

```
# configure the classification job
classification_job = automl.classification(
    compute="aml-cluster",
    experiment_name="auto-ml-class-dev",
    training_data=my_training_data_input,
    target_column_name="Diabetic",
    primary_metric="accuracy",
    n_cross_validations=5,
    enable_model_explainability=True
)
```

> **Note**: AutoML needs a MLTable data asset as input. In the example, `my_training_data_input` refers to a MLTable data asset created in the Azure Machine Learning workspace.

Specify the primary metric

One of the most important settings you must specify is the **primary_metric**. AutoML uses the primary metric to rank all trained models and select the best one. Azure Machine Learning supports a set of named metrics for each task type.

To retrieve the list of metrics available when you want to train a classification model, you can use the **ClassificationPrimaryMetrics** function as shown here:

```
from azure.ai.ml.automl import ClassificationPrimaryMetrics

list(ClassificationPrimaryMetrics)
```

> **Tip**: You can find a full list of primary metrics and their definitions in evaluate automated machine learning experiment results.

Set the limits

Each model AutoML trains consumes compute resources. To control costs and training time, you can set limits on an AutoML job using `set_limits()`.

There are several options to set limits to an AutoML experiment:

- `timeout_minutes`: Number of minutes after which the complete AutoML experiment is terminated.
- `trial_timeout_minutes`: Maximum number of minutes one trial can take.
- `max_trials`: Maximum number of trials, or models that are trained.
- `enable_early_termination`: Whether to end the experiment if the score isn't improving in the short term.

```
classification_job.set_limits(
    timeout_minutes=60,
    trial_timeout_minutes=20,
    max_trials=5,
```

```
    enable_early_termination=True,
)
```

To save time, you can also run multiple trials in parallel. When you use a compute cluster, you can have as many parallel trials as you have nodes. The maximum number of parallel trials is therefore related to the maximum number of nodes your compute cluster has. If you want to set the maximum number of parallel trials to be less than the maximum number of nodes, you can use `max_concurrent_trials`.

Set the training properties

AutoML tries various combinations of featurization and algorithms to train a machine learning model. If you already know that certain algorithms aren't well-suited for your data, you can exclude (or include) a subset of the available algorithms.

You can also choose whether you want to allow AutoML to use ensemble models.

Submit an AutoML experiment

You can submit an AutoML job with the following code:

```
# submit the AutoML job
returned_job = ml_client.jobs.create_or_update(
    classification_job
)
```

You can monitor AutoML job runs in the Azure Machine Learning studio. To get a direct link to the AutoML job by running the following code:

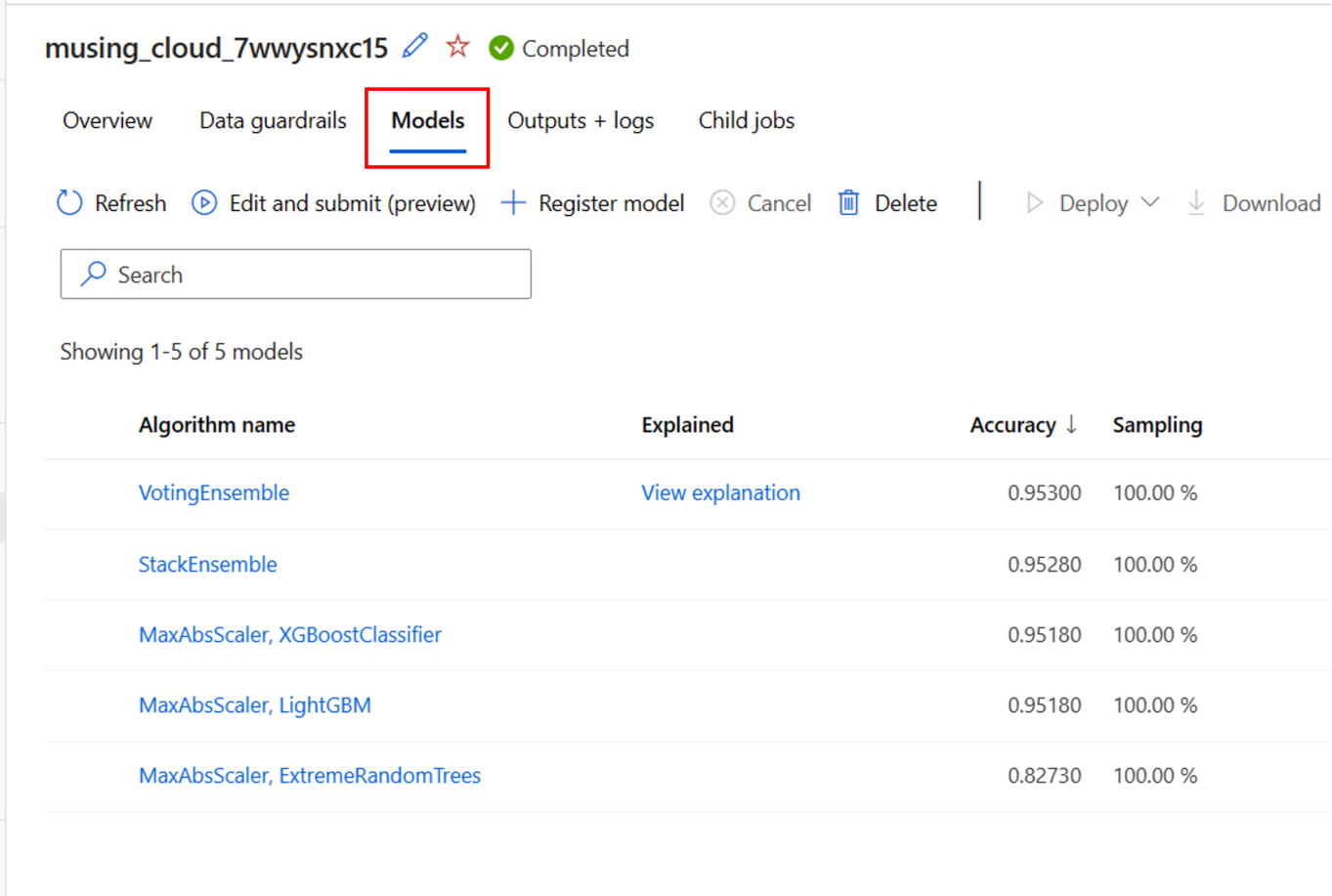
```
aml_url = returned_job.studio_url
print("Monitor your job at", aml_url)
```

Evaluate and compare models

When an automated machine learning (AutoML) experiment finishes, you want to review the models that were trained and decide which one performed best.

In the Azure Machine Learning studio, you can select an AutoML experiment to explore its details.

On the **Overview** page of the AutoML experiment run, you can review the input data asset and the summary of the best model. To explore all models that were trained, you can select the **Models** tab:



Explore preprocessing steps

When you enabled featurization for your AutoML experiment, data guardrails are automatically applied too. The three data guardrails that are supported for classification models are:

- Class balancing detection.
- Missing feature values imputation.
- High cardinality feature detection.

Each of these data guardrails show one of three possible states:

- **Passed**: No problems were detected and no action is required.
- **Done**: Changes were applied to your data. You should review the changes AutoML made to your data.
- **Alerted**: An issue was detected but couldn't be fixed. You should review the data to fix the issue.

Next to data guardrails, AutoML can apply scaling and normalization techniques to each model that is trained. You can review the technique applied in the list of models under **Algorithm name**.

For example, the algorithm name of a model listed can be `MaxAbsScaler, LightGBM`. `MaxAbsScaler` refers to a scaling technique where each feature is scaled by its maximum absolute value. `LightGBM` refers to the classification algorithm used to train the model.

Retrieve the best run and its model

When you're reviewing the models in AutoML, you can easily identify the best run based on the primary metric you specified. In the Azure Machine Learning studio, the models are automatically sorted to show the best performing model at the top.

In the **Models** tab of the AutoML experiment, you can **edit the columns** if you want to show other metrics in the same overview. By creating a more comprehensive overview that includes various metrics, it can be easier to compare models.

To explore a model even further, you can generate explanations for each model that was trained. When configuring an AutoML experiment, you can specify that explanations should be generated for the best performing model. If however, you're interested in the interpretability of another model, you can select the model in the overview and select **Explain model**.

> **Note**: Explaining a model is an approximation to the model's interpretability. Specifically, explanations will estimate the relative importance of features on the target feature (what the model is trained to predict). Learn more about model interpretability.

> **Tip**: Learn more about how to evaluate AutoML runs.

AutoML gives you a strong starting point — it searches broadly and surfaces the best algorithm and preprocessing combination for your data. But sometimes you want to go further: adjust hyperparameters, engineer custom features, or test an approach that AutoML doesn't cover. That's where notebooks come in.

Configure MLflow for model tracking in notebooks

Working in a notebook lets you experiment interactively and iterate quickly. To make that experimentation meaningful, you need to track what you try. Without tracking, it's easy to lose sight of which configuration produced which result.

MLflow is an open-source library for tracking and managing your machine learning experiments. In particular, **MLflow Tracking** is a component of MLflow that logs everything about the model you're training, such as **parameters**, **metrics**, and **artifacts**. This means you can compare your notebook runs directly against the models AutoML trained, all in one place.

To use MLflow in notebooks in the Azure Machine Learning workspace, you need to install the necessary libraries and set Azure Machine Learning as the tracking store.

Configure MLflow in notebooks

You can create and edit notebooks within Azure Machine Learning or on a local device.

Use Azure Machine Learning notebooks

Within the Azure Machine Learning workspace, you can create notebooks and connect the notebooks to an Azure Machine Learning managed **compute instance**.

When you're running a notebook on a compute instance, MLflow is already configured, and ready to be used.

To verify that the necessary packages are installed, you can run the following code:

```
pip show mlflow
pip show azureml-mlflow
```

The `mlflow` package is the open-source library. The `azureml-mlflow` package contains the integration code of Azure Machine Learning with MLflow.

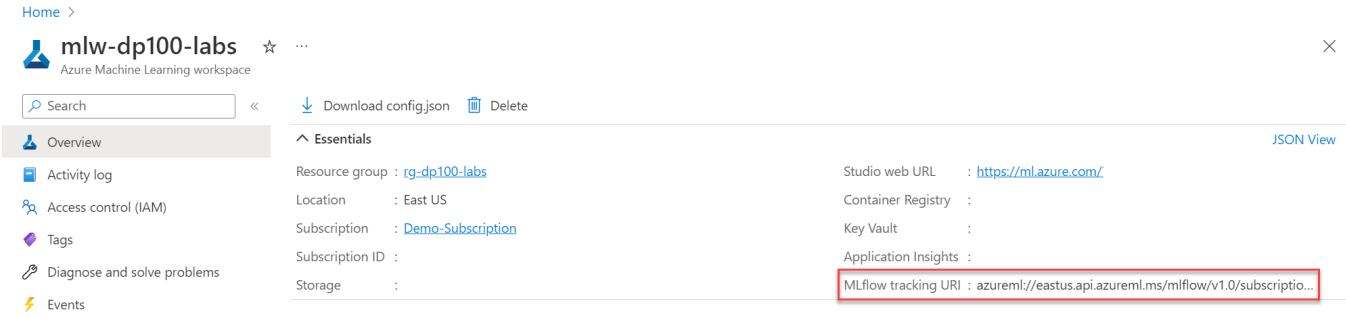
Use MLflow on a local device

When you prefer working in notebooks on a local device, you can also make use of MLflow. You need to configure MLflow by completing the following steps:

1. Install the `mlflow` and `azureml-mlflow` package.

```
pip install mlflow
pip install azureml-mlflow
```

2. Navigate to the Azure Machine Learning studio.
3. Select the name of the workspace you're working on in the top right corner of the studio.
4. Select **View all properties in Azure portal**. A new tab opens to take you to the Azure Machine Learning service in the Azure portal.
5. Copy the value of the **MLflow tracking URI**.



6. Use the following code in your local notebook to configure MLflow to point to the Azure Machine Learning workspace, and set it to the workspace tracking URI.

```
mlflow.set_tracking_uri = "MLFLOW-TRACKING-URI"
```

> **Tip**: Learn about alternative approaches to set up the tracking environment when working on a local device. For example, you can also use the Azure Machine Learning SDK v2 for Python, together with the workspace configuration file, to set the tracking URI.

When you configured MLflow to track your model's results and store it in your Azure Machine Learning workspace, you're ready to experiment in a notebook.

Train and track models in notebooks

As a data scientist, you use notebooks to experiment and train models. To group model training results, you use **experiments**. To track model metrics with MLflow when training a model in a notebook, you can use

MLflow's logging capabilities.

Create a MLflow experiment

You can create a MLflow experiment, which allows you to group runs. If you don't create an experiment, MLflow assumes the default experiment with name `Default`.

To create an experiment, run the following command in a notebook:

```
import mlflow

mlflow.set_experiment(experiment_name="heart-condition-classifier")
```

Log results with MLflow

Now, you're ready to train your model. To start a run tracked by MLflow, you use `start_run()`. Next, to track the model, you can:

- Enable **autologging**.
- Use **custom logging**.

Enable autologging

MLflow supports automatic logging for popular machine learning libraries. When you enable autologging, MLflow instructs your framework to log metrics, parameters, artifacts, and models automatically. You don't need to specify what to log as the framework decides what's relevant.

You can turn on autologging by calling `mlflow.autolog()` before your training code. You can also use the framework-specific method, such as `mlflow.xgboost.autolog()`, for more granular control.

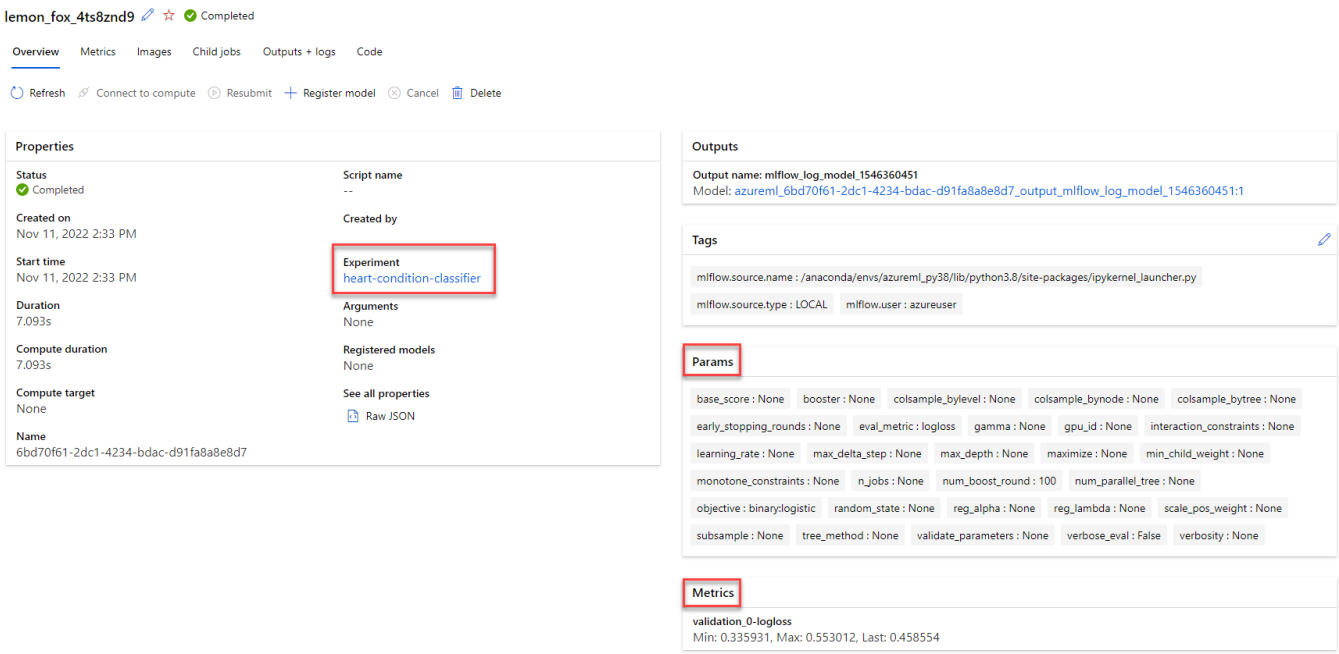
> **Tip**: Find a list of all supported frameworks for autologging in the official MLflow documentation.

A notebook cell that trains and tracks a classification model using autologging can be similar to the following code example:

```
from xgboost import XGBClassifier

with mlflow.start_run():
    mlflow.autolog()

    model = XGBClassifier(eval_metric="logloss")
    model.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
```

As soon as `mlflow.xgboost.autolog()` is called, MLflow starts a run within an experiment in Azure Machine Learning to start tracking the experiment's run.

When the job completes, you can review all logged metrics in the studio.



Use custom logging

Additionally, you can manually log your model with MLflow. Manually logging models is helpful when you want to log supplementary or custom information that isn't logged through autologging.

> **Note**: You can choose to only use custom logging, or use custom logging in combination with autologging.

Common functions used with custom logging are:

- `mlflow.log_param()`: Logs a single key-value parameter. Use this function for an input parameter you want to log.
- `mlflow.log_metric()`: Logs a single key-value metric. Value must be a number. Use this function for any output you want to store with the run.
- `mlflow.log_figure()`: Logs a matplotlib figure directly as an artifact.
- `mlflow.log_image()`: Logs a numpy or PIL image as an artifact.
- `mlflow.log_artifact()`: Logs any existing file as an artifact.
- `mlflow.log_model()`: Logs a model. Use this function to create a MLflow model, which can include a custom signature, environment, and input examples.

> **Tip**: Learn more about how to track models with MLflow by exploring the official MLflow documentation, or the Azure Machine Learning documentation

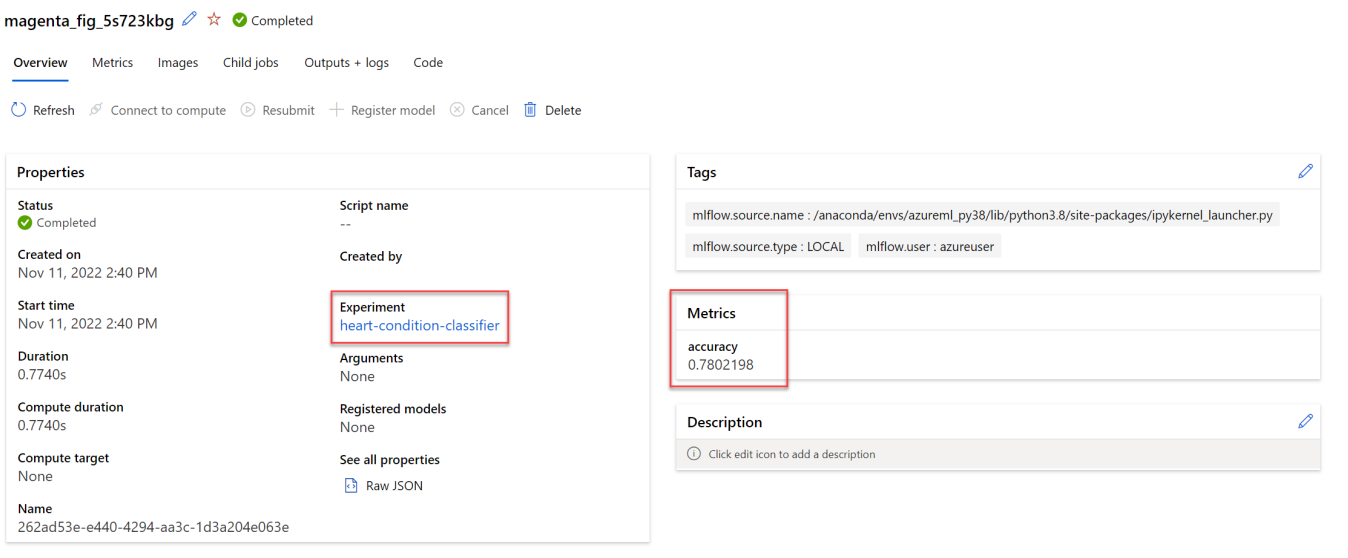
To use custom logging in a notebook, start a run and log any metric you want:

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

with mlflow.start_run():
    model = XGBClassifier(eval_metric="logloss")
    model.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
    y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
mlflow.log_metric("accuracy", accuracy)
```

Custom logging gives you more flexibility, but also creates more work as you have to define any parameter, metric, or artifact you want to log.

When the job completes, you can review all logged metrics in the studio.

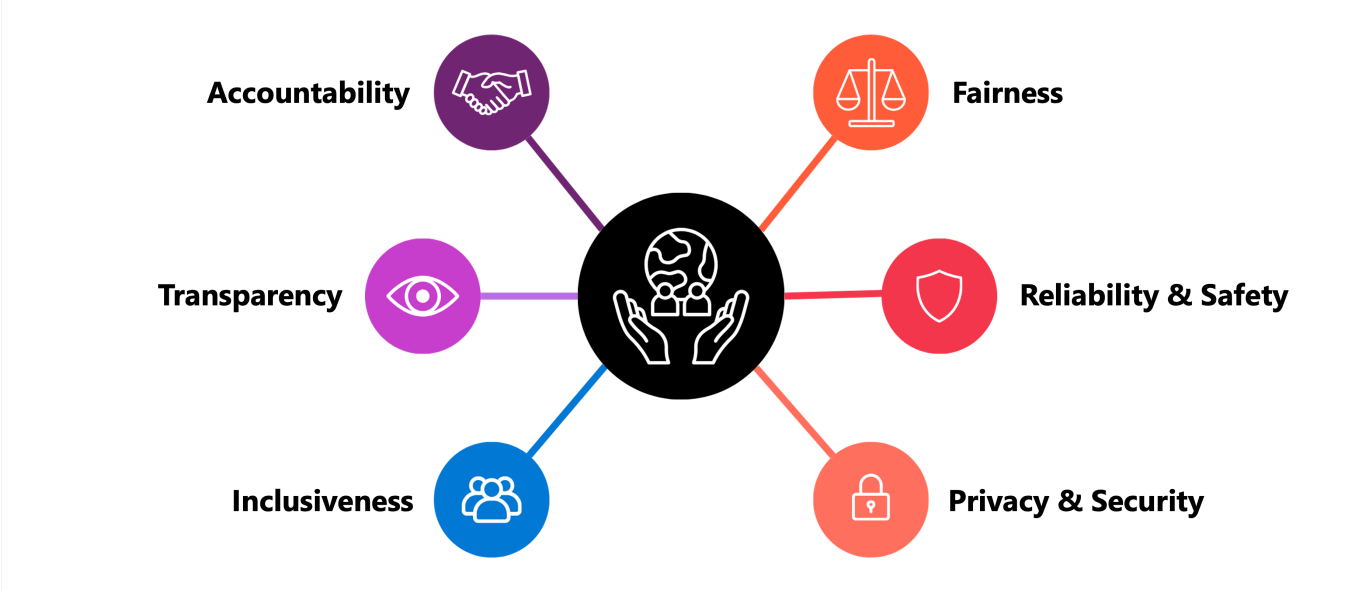


Evaluate models with the Responsible AI dashboard

After training a model in a notebook, you want to evaluate it — not just for accuracy, but also for fairness, transparency, and reliability. Azure Machine Learning's **Responsible AI dashboard** brings these evaluations together in one interactive view.

Why responsible AI matters

Models are often used when making consequential decisions. Whatever your model predicts, you should consider Microsoft's six **Responsible AI principles**:



- **Fairness**: Ensure your model provides equitable outcomes by testing for and mitigating harmful bias across groups.
- **Reliability & Safety**: Build, test, and monitor your model so it performs consistently and prevents unsafe behavior.
- **Privacy & Security**: Protect user data through minimal collection and responsible data-handling practices.
- **Inclusiveness**: Design and evaluate systems so people of diverse abilities and backgrounds can use them effectively.
- **Transparency**: Communicate clearly how your model works and how its outputs should be interpreted.
- **Accountability**: Assign human oversight so decisions influenced by AI remain traceable and governed.

Create a Responsible AI dashboard

To generate a Responsible AI (RAI) dashboard, you create a **pipeline** using Azure Machine Learning's built-in RAI components. The pipeline must:

1. Start with the `RAI Insights dashboard constructor`.
2. Include one or more **RAI tool components** for the insights you need.
3. End with `Gather RAI Insights dashboard` to collect everything into one dashboard.

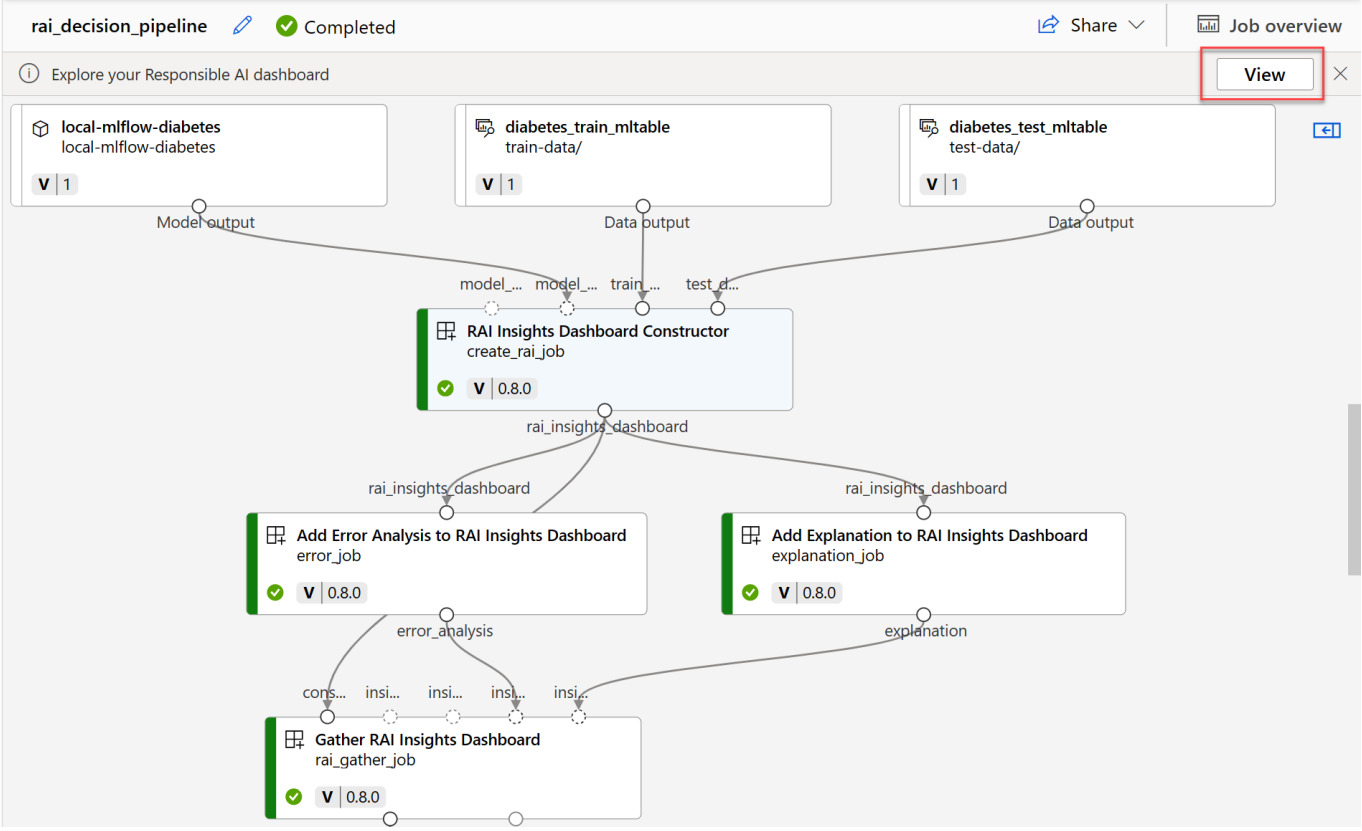
The available RAI tool components are:

- `Add Explanation to RAI Insights dashboard`: Shows how much each feature influences the model's predictions.
- `Add Error Analysis to RAI Insights dashboard`: Identifies subgroups of data where the model makes more errors.
- `Add Counterfactuals to RAI Insights dashboard`: Explores how changes in input would change the model's output.
- `Add Causal to RAI Insights dashboard`: Uses historical data to estimate the causal effect of features on outcomes.

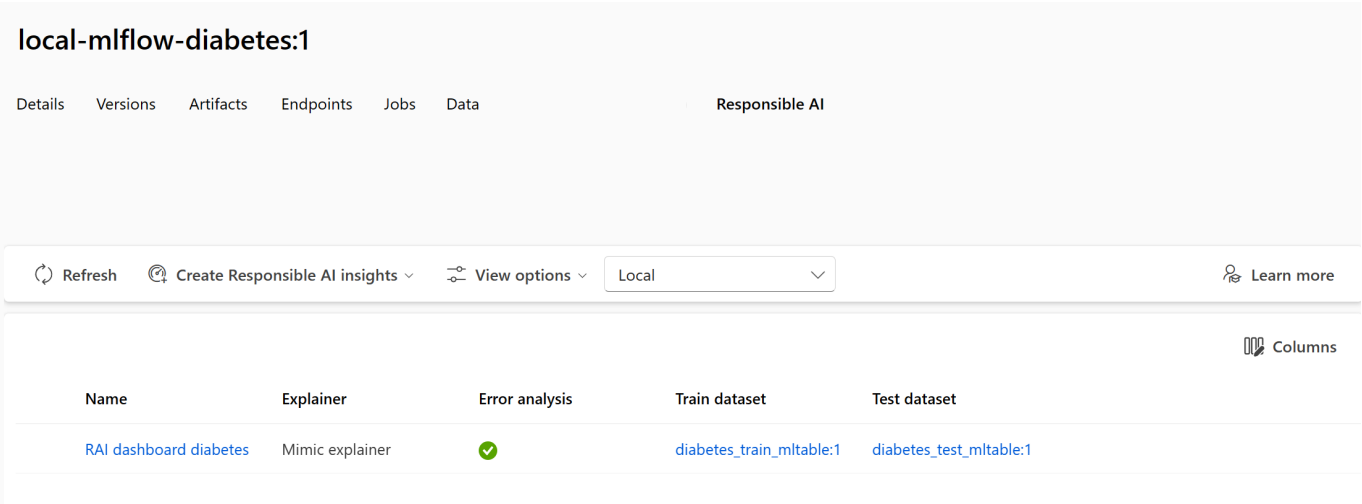
You can build this pipeline using the Python SDK, the CLI, or the no-code experience in Azure Machine Learning studio.

Explore the dashboard

Once the pipeline completes, you can open the **Responsible AI dashboard** from the pipeline overview, or from the **Responsible AI** tab of the registered model in the studio.

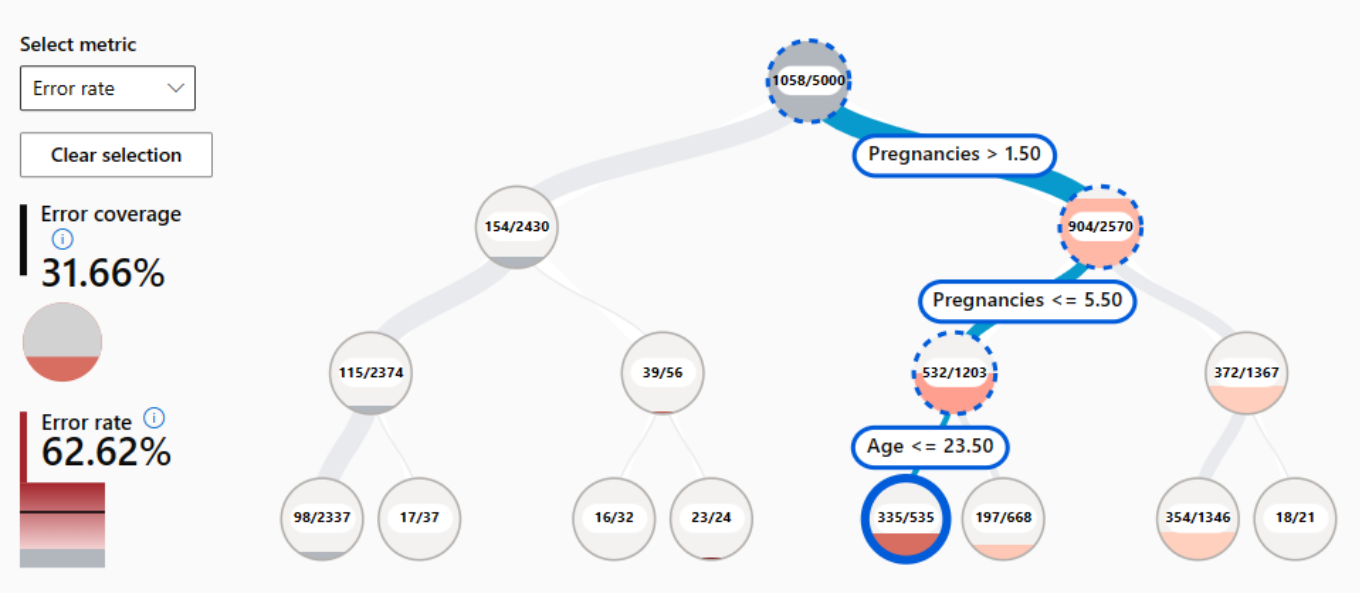


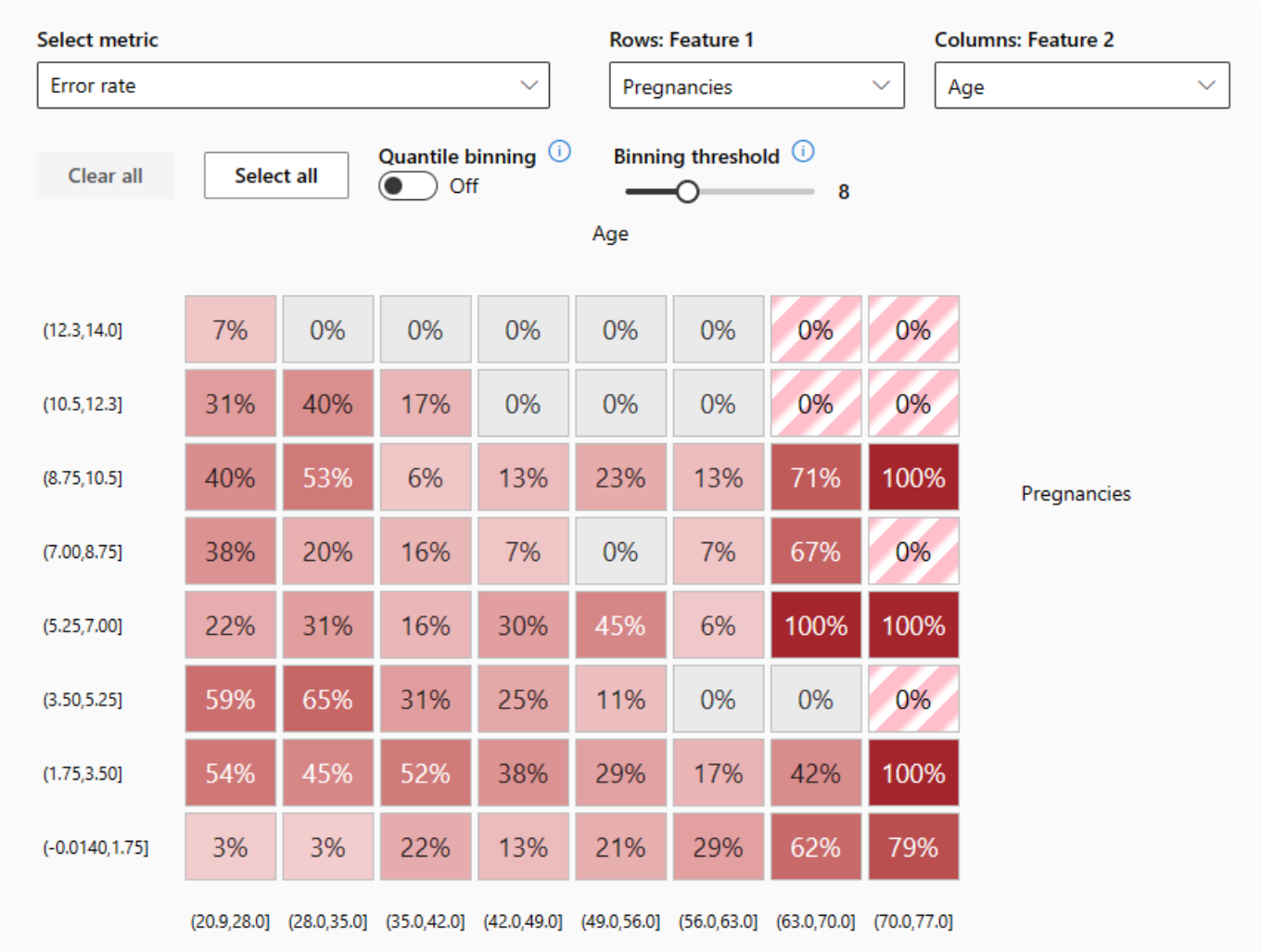
Alternatively, find the dashboard in the **Responsible AI** tab of the registered model.



Error analysis

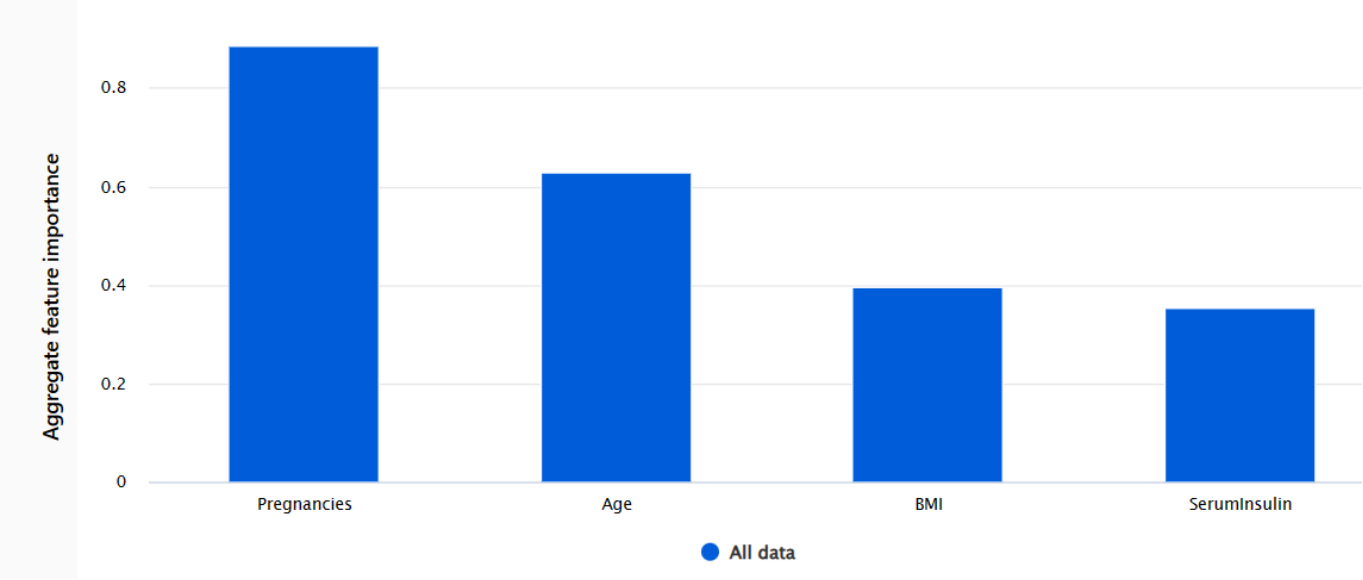
Error analysis shows how prediction errors are distributed across your dataset. You can use the **error tree map** to find combinations of subgroups with higher error rates, or the **error heat map** to see errors across one or two features.

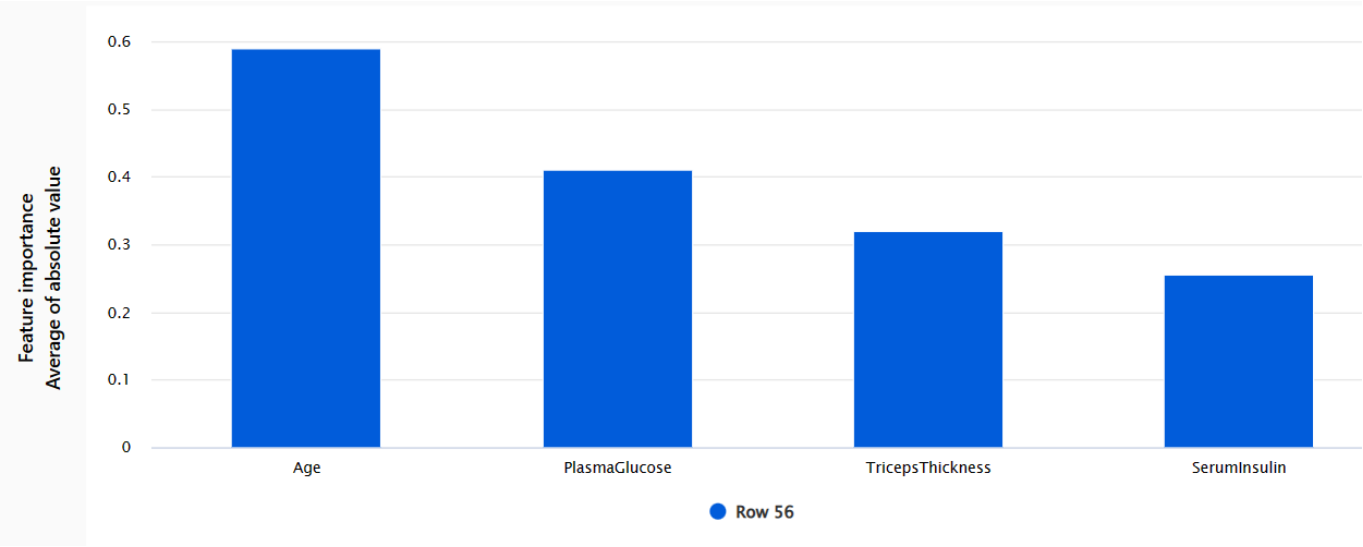




Explanation

Feature importance tells you how much each input feature influences the model's predictions. **Aggregate feature importance** shows overall influence across your test data; **individual feature importance** shows the influence for a single prediction.





Counterfactuals

Counterfactuals let you ask *what-if* questions: if this input were different, would the prediction change? Select a data point and a desired outcome to explore which minimal changes would flip the model's prediction.

Causal analysis

Causal analysis estimates the average effect of a feature on an outcome using statistical techniques. Use it to understand which interventions are likely to improve outcomes — either across a population or for individual data points.

> **Tip**: Learn more about the Responsible AI dashboard in Azure Machine Learning.

Find the best classification model with Azure Machine Learning

Determining the right algorithm and preprocessing transformations for model training can involve a lot of guesswork and experimentation.

In this exercise, you'll experiment to find the best classification model in three phases:

- Start by using automated machine learning to determine the optimal algorithm and preprocessing steps for a model by performing multiple training runs in parallel.
- Continue experimenting by training a classification model in an interactive notebook and track your work with MLflow.

Before you start

You'll need an Azure subscription in which you have administrative-level access.

Provision an Azure Machine Learning workspace

An Azure Machine Learning *workspace* provides a central place for managing all resources and assets you need to train and manage your models. You can interact with the Azure Machine Learning workspace through the studio, Python SDK, and Azure CLI.

You'll use the Azure CLI to provision the workspace and necessary compute, and you'll use the Python SDK in interactive notebooks to train machine learning models.

Create the workspace and compute resources

To create the Azure Machine Learning workspace, a compute instance, and a compute cluster, you'll use the Azure CLI. All necessary commands are grouped in a Shell script for you to execute.

1. In a browser, open the Azure portal at `https://portal.azure.com/`, signing in with your Microsoft account.

2. Select the [>_] (*Cloud Shell*) button at the top of the page to the right of the search box. This opens a Cloud Shell pane at the bottom of the portal.

3. Select **Bash** if asked. The first time you open the cloud shell, you will be asked to choose the type of shell you want to use (*Bash* or *PowerShell*).

4. Check that the correct subscription is specified and that **No storage account required** is selected. Select **Apply**.

5. In the terminal, enter the following commands to clone this repo:

   ```
   rm -r mslearn-mlops -f
   git clone https://github.com/MicrosoftLearning/mslearn-mlops.git mslearn-mlops
   ```

   > Use `SHIFT + INSERT` to paste your copied code into the Cloud Shell.

6. After the repo has been cloned, enter the following commands to change to the folder for this lab and run the **setup.sh** script it contains:

   ```
   cd mslearn-mlops/infra
   ./setup.sh
   ```

> Ignore any (error) messages that say that the extensions were not installed.

7. Wait for the script to complete - this typically takes around 5-10 minutes.

   Troubleshooting tip: Workspace creation error

   If you receive an error when running the setup script through the CLI, you need to provision the resources manually:

   1. In the Azure portal home page, select **+ Create a resource**.
   2. Search for *machine learning* and then select **Azure Machine Learning**. Select **Create**.
   3. Create a new Azure Machine Learning resource with the following settings:
      - **Subscription**: *Your Azure subscription*
      - **Resource group**: rg-ai300-labs
      - **Workspace name**: mlw-ai300-labs
      - **Region**: *Select the geographical region closest to you*
      - **Storage account**: *Note the default new storage account that will be created for your workspace*
      - **Key vault**: *Note the default new key vault that will be created for your workspace*
      - **Application insights**: *Note the default new application insights resource that will be created for your workspace*
      - **Container registry**: None (*one will be created automatically the first time you deploy a model to a container*)
   4. Select **Review + create** and wait for the workspace and its associated resources to be created - this typically takes around 5 minutes.
   5. Select **Go to resource** and in its **Overview** page, select **Launch studio**. Another tab will open in your browser to open the Azure Machine Learning studio.
   6. Close any pop-ups that appear in the studio.
   7. Within the Azure Machine Learning studio, navigate to the **Compute** page and select **+ New** under the **Compute instances** tab.
   8. Give the compute instance a unique name and then select **Standard_DS11_v2** as the virtual machine size.
   9. Select **Review + create** and then select **Create**.
   10. Next, select the **Compute clusters** tab and select **+ New**.
   11. Choose the same region as the one where you created your workspace and then select **Standard_DS11_v2** as the virtual machine size. Select **Next**
   12. Give the cluster a unique name and then select **Create**.

Clone the lab materials

When you’ve created the workspace and necessary compute resources, you can open the Azure Machine Learning studio and clone the lab materials into the workspace.

1. In the Azure portal, navigate to the Azure Machine Learning workspace named **mlw-ai300-...**.

2. Select the Azure Machine Learning workspace, and in its **Overview** page, select **Launch studio**. Another tab will open in your browser to open the Azure Machine Learning studio.

3. Close any pop-ups that appear in the studio.

4. Within the Azure Machine Learning studio, navigate to the **Compute** page and verify that the compute instance and cluster you created in the previous section exist. The compute instance should be running, the cluster should be idle and have 0 nodes running.

5. In the **Compute instances** tab, find your compute instance, and select the **Terminal** application.

6. In the terminal, install the Python SDK on the compute instance by running the following commands in the terminal:

```
pip uninstall azure-ai-ml
pip install azure-ai-ml
```

> Ignore any (error) messages that say that the packages couldn't be found and uninstalled.

7. Run the following command to clone a Git repository containing notebooks, data, and other files to your workspace:

```
git clone https://github.com/MicrosoftLearning/mslearn-mlops.git mslearn-mlops
```

8. When the command has completed, in the **Files** pane, click ↻ to refresh the view and verify that a new **Users/*your-user-name*/mslearn-mlops** folder has been created.

Train a classification model with automated machine learning

Now that you have all the necessary resources, you can run the notebook to configure and submit the Automated Machine Learning job.

1. Open the **experimentation/Classification with Automated Machine Learning.ipynb** notebook.

> Select **Authenticate** and follow the necessary steps if a notification appears asking you to authenticate.

2. Verify that the notebook uses the **Python 3.10 - AzureML** kernel.

3. Run all cells in the notebook.

   A new job will be created in the Azure Machine Learning workspace. The job tracks the inputs defined in the job configuration, the data asset used, and the outputs like metrics to evaluate the models.

   Note that the Automated Machine Learning jobs contains child jobs, which represent individual models that have been trained and other tasks needed to execute.

4. Go to **Jobs** and select the **auto-ml-class-dev** experiment.

5. Select the job under the **Display name** column.

6. Wait for its status to change to **Completed**.

7. When the Automate Machine Learning job status has changed to **Completed**, explore the job details in the studio:

    - The **Data guardrails** tab shows whether your training data had any issues.
    - The **Models + child jobs** tab will show all models that have been trained.

Track model training with MLflow

Now that you have done your initial exploration, you can also take full control of model training by running a notebook. You can use MLflow when training models in a notebook to track parameters, metrics, and other artefacts.

1. Navigate back to the **Files** pane.

2. Open the **experimentation/Track model training with MLflow.ipynb** notebook.

3. Verify that the notebook uses the **Python 3.10 - AzureML** kernel.

4. Run all cells in the notebook.

5. Review the new job that's created every time you train a model.

    > **Note:** When you train a model, the cell's output will show a link to the job run. If the link returns an error, you can still review the job run by selecting **Jobs** on the left side panel.

Delete Azure resources

When you finish exploring Azure Machine Learning, you should delete the resources you've created to avoid unnecessary Azure costs.

1. Close the Azure Machine Learning studio tab and return to the Azure portal.
2. In the Azure portal, on the **Home** page, select **Resource groups**.
3. Select the **rg-ai300-...** resource group.
4. At the top of the **Overview** page for your resource group, select **Delete resource group**.
5. Enter the resource group name to confirm you want to delete it, and select **Delete**.

Perform hyperparameter tuning with Azure Machine Learning

Introduction

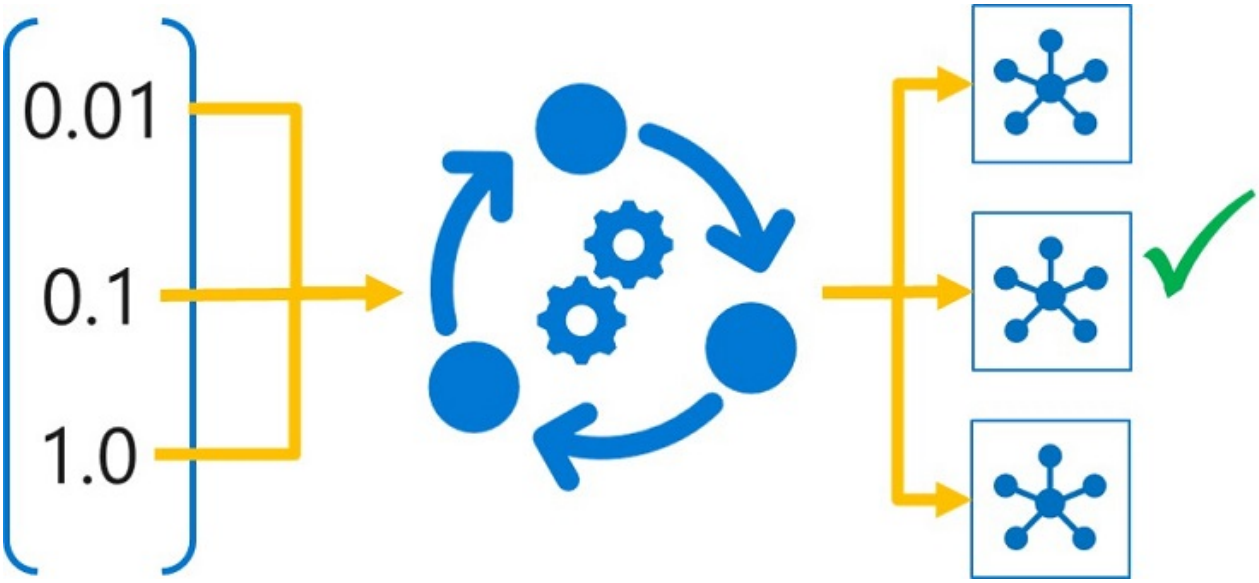
In machine learning, models are trained to predict unknown labels for new data based on correlations between known labels and features found in the training data. Depending on the algorithm used, you may need to specify **hyperparameters** to configure how the model is trained.

For example, the *logistic regression* algorithm uses a *regularization rate* hyperparameter to counteract overfitting; and deep learning techniques for convolutional neural networks (CNNs) use hyperparameters like *learning rate* to control how weights are adjusted during training, and *batch size* to determine how many data items are included in each training batch.

> **Note**: Machine Learning is an academic field with its own particular terminology. Data scientists refer to the values determined from the training features as *parameters*, so a different term is required for values that are used to configure training behavior but which are ***not*** derived from the training data - hence the term *hyperparameter*.

The choice of hyperparameter values can significantly affect the resulting model, making it important to select the best possible values for your particular data and predictive performance goals.

Tuning hyperparameters



Hyperparameter tuning is accomplished by training the multiple models, using the same algorithm and training data but different hyperparameter values. The resulting model from each training run is then evaluated to determine the performance metric for which you want to optimize (for example, *accuracy*), and the best-performing model is selected.

In Azure Machine Learning, you can tune hyperparameters by submitting a script as a **sweep job**. A sweep job will run a **trial** for each hyperparameter combination to be tested. Each trial uses a training script with parameterized hyperparameter values to train a model, and logs the target performance metric achieved by the trained model.

Define a search space

The set of hyperparameter values tried during hyperparameter tuning is known as the **search space**. The definition of the range of possible values that can be chosen depends on the type of hyperparameter.

Discrete hyperparameters

Some hyperparameters require *discrete* values - in other words, you must select the value from a particular *finite* set of possibilities. You can define a search space for a discrete parameter using a **Choice** from a list of explicit values, which you can define as a Python **list** (`Choice(values=[10,20,30])`), a **range** (`Choice(values=range(1,10))`), or an arbitrary set of comma-separated values (`Choice(values=(30,50,100))`)

You can also select discrete values from any of the following discrete distributions:

- `QUniform(min_value, max_value, q)`: Returns a value like round(Uniform(min_value, max_value) / q) * q
- `QLogUniform(min_value, max_value, q)`: Returns a value like round(exp(Uniform(min_value, max_value)) / q) * q
- `QNormal(mu, sigma, q)`: Returns a value like round(Normal(mu, sigma) / q) * q
- `QLogNormal(mu, sigma, q)`: Returns a value like round(exp(Normal(mu, sigma)) / q) * q

Continuous hyperparameters

Some hyperparameters are *continuous* - in other words you can use any value along a scale, resulting in an *infinite* number of possibilities. To define a search space for these kinds of value, you can use any of the following distribution types:

- `Uniform(min_value, max_value)`: Returns a value uniformly distributed between min_value and max_value
- `LogUniform(min_value, max_value)`: Returns a value drawn according to exp(Uniform(min_value, max_value)) so that the logarithm of the return value is uniformly distributed
- `Normal(mu, sigma)`: Returns a real value that's normally distributed with mean mu and standard deviation sigma
- `LogNormal(mu, sigma)`: Returns a value drawn according to exp(Normal(mu, sigma)) so that the logarithm of the return value is normally distributed

Defining a search space

To define a search space for hyperparameter tuning, create a dictionary with the appropriate parameter expression for each named hyperparameter.

For example, the following search space indicates that the `batch_size` hyperparameter can have the value 16, 32, or 64, and the `learning_rate` hyperparameter can have any value from a normal distribution with a mean of 10 and a standard deviation of 3.

```
from azure.ai.ml.sweep import Choice, Normal

command_job_for_sweep = job(
    batch_size=Choice(values=[16, 32, 64]),
    learning_rate=Normal(mu=10, sigma=3),
)
```

Configure a sampling method

The specific values used in a hyperparameter tuning run, or **sweep job**, depend on the type of **sampling** used.

There are three main sampling methods available in Azure Machine Learning:

- **Grid sampling**: Tries every possible combination.
- **Random sampling**: Randomly chooses values from the search space.
    - **Sobol**: Adds a seed to random sampling to make the results reproducible.

- **Bayesian sampling**: Chooses new values based on previous results.

> **Note**: Sobol is a variation of random sampling.

Grid sampling

Grid sampling can only be applied when all hyperparameters are discrete, and is used to try every possible combination of parameters in the search space.

For example, in the following code example, grid sampling is used to try every possible combination of discrete *batch_size* and *learning_rate* value:

```
from azure.ai.ml.sweep import Choice

command_job_for_sweep = command_job(
    batch_size=Choice(values=[16, 32, 64]),
    learning_rate=Choice(values=[0.01, 0.1, 1.0]),
)

sweep_job = command_job_for_sweep.sweep(
    sampling_algorithm = "grid",
    ...
)
```

Random sampling

Random sampling is used to randomly select a value for each hyperparameter, which can be a mix of discrete and continuous values as shown in the following code example:

```
from azure.ai.ml.sweep import Normal, Uniform

command_job_for_sweep = command_job(
    batch_size=Choice(values=[16, 32, 64]),
    learning_rate=Normal(mu=10, sigma=3),
)

sweep_job = command_job_for_sweep.sweep(
    sampling_algorithm = "random",
    ...
)
```

Sobol

You may want to be able to reproduce a random sampling sweep job. If you expect that you do, you can use Sobol instead. Sobol is a type of random sampling that allows you to use a seed. When you add a seed, the sweep job can be reproduced, and the search space distribution is spread more evenly.

The following code example shows how to use Sobol by adding a seed and a rule, and using the `RandomSamplingAlgorithm` class:

```
from azure.ai.ml.sweep import RandomSamplingAlgorithm

sweep_job = command_job_for_sweep.sweep(
    sampling_algorithm = RandomSamplingAlgorithm(seed=123, rule="sobol"),
    ...
)
```

Bayesian sampling

Bayesian sampling chooses hyperparameter values based on the Bayesian optimization algorithm, which tries to select parameter combinations that will result in improved performance from the previous selection. The following code example shows how to configure Bayesian sampling:

```
from azure.ai.ml.sweep import Uniform, Choice

command_job_for_sweep = job(
    batch_size=Choice(values=[16, 32, 64]),
    learning_rate=Uniform(min_value=0.05, max_value=0.1),
)

sweep_job = command_job_for_sweep.sweep(
    sampling_algorithm = "bayesian",
    ...
)
```

You can only use Bayesian sampling with **choice**, **uniform**, and **quniform** parameter expressions.

Configure early termination

Hyperparameter tuning helps you fine-tune your model and select the hyperparameter values that will make your model perform best.

For you to find the best model, however, can be a never-ending conquest. You always have to consider whether it's worth the time and expense of testing new hyperparameter values to find a model that may perform better.

Each trial in a sweep job, a new model is trained with a new combination of hyperparameter values. If training a new model doesn't result in a significantly better model, you may want to stop the sweep job and use the model that performed best so far.

When you configure a sweep job in Azure Machine Learning, you can also set a maximum number of trials. A more sophisticated approach may be to stop a sweep job when newer models don't produce significantly better results. To stop a sweep job based on the performance of the models, you can use an **early termination policy**.

When to use an early termination policy

Whether you want to use an early termination policy may depend on the search space and sampling method you're working with.

For example, you may choose to use a *grid sampling* method over a *discrete* search space that results in a maximum of six trials. With six trials, a maximum of six models will be trained and an early termination policy may be unnecessary.

An early termination policy can be especially beneficial when working with continuous hyperparameters in your search space. Continuous hyperparameters present an unlimited number of possible values to choose from. You'll most likely want to use an early termination policy when working with continuous hyperparameters and a random or Bayesian sampling method.

Configure an early termination policy

There are two main parameters when you choose to use an early termination policy:

- `evaluation_interval`: Specifies at which interval you want the policy to be evaluated. Every time the primary metric is logged for a trial counts as an interval.
- `delay_evaluation`: Specifies when to start evaluating the policy. This parameter allows for at least a minimum of trials to complete without an early termination policy affecting them.

New models may continue to perform only slightly better than previous models. To determine the extent to which a model should perform better than previous trials, there are three options for early termination:

- **Bandit policy**: Uses a `slack_factor` (relative) or `slack_amount`(absolute). Any new model must perform within the slack range of the best performing model.
- **Median stopping policy**: Uses the median of the averages of the primary metric. Any new model must perform better than the median.
- **Truncation selection policy**: Uses a `truncation_percentage`, which is the percentage of lowest performing trials. Any new model must perform better than the lowest performing trials.

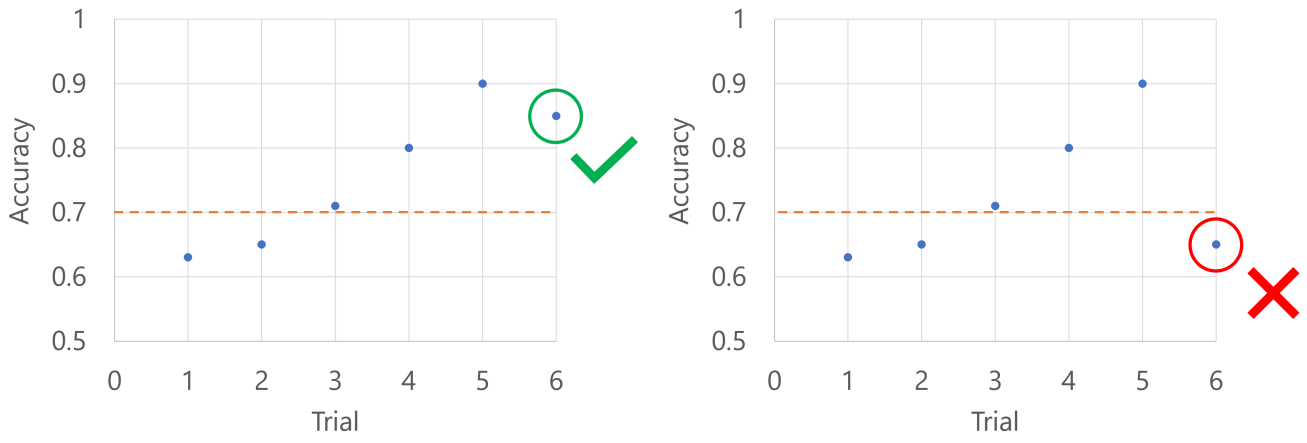
Bandit policy

You can use a bandit policy to stop a trial if the target performance metric underperforms the best trial so far by a specified margin.

For example, the following code applies a bandit policy with a delay of five trials, evaluates the policy at every interval, and allows an absolute slack amount of 0.2.

```
from azure.ai.ml.sweep import BanditPolicy

sweep_job.early_termination = BanditPolicy(
    slack_amount = 0.2,
    delay_evaluation = 5,
    evaluation_interval = 1
)
```

Imagine the primary metric is the accuracy of the model. When after the first five trials, the best performing model has an accuracy of 0.9, any new model needs to perform better than (0.9-0.2) or 0.7. If the new model's accuracy is higher than 0.7, the sweep job will continue. If the new model has an accuracy score lower than 0.7, the policy will terminate the sweep job.



You can also apply a bandit policy using a slack *factor*, which compares the performance metric as a ratio rather than an absolute value.

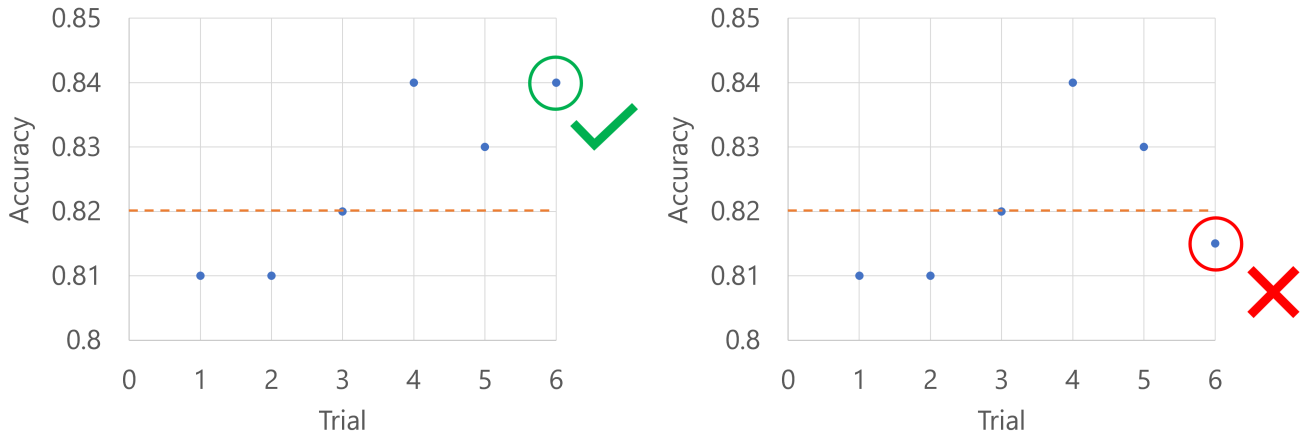
Median stopping policy

A median stopping policy abandons trials where the target performance metric is worse than the median of the running averages for all trials.

For example, the following code applies a median stopping policy with a delay of five trials and evaluates the policy at every interval.

```
from azure.ai.ml.sweep import MedianStoppingPolicy

sweep_job.early_termination = MedianStoppingPolicy(
    delay_evaluation = 5,
    evaluation_interval = 1
)
```

Imagine the primary metric is the accuracy of the model. When the accuracy is logged for the sixth trial, the metric needs to be higher than the median of the accuracy scores so far. Suppose the median of the accuracy scores so far is 0.82. If the new model's accuracy is higher than 0.82, the sweep job will continue. If the new model has an accuracy score lower than 0.82, the policy will stop the sweep job, and no new models will be trained.



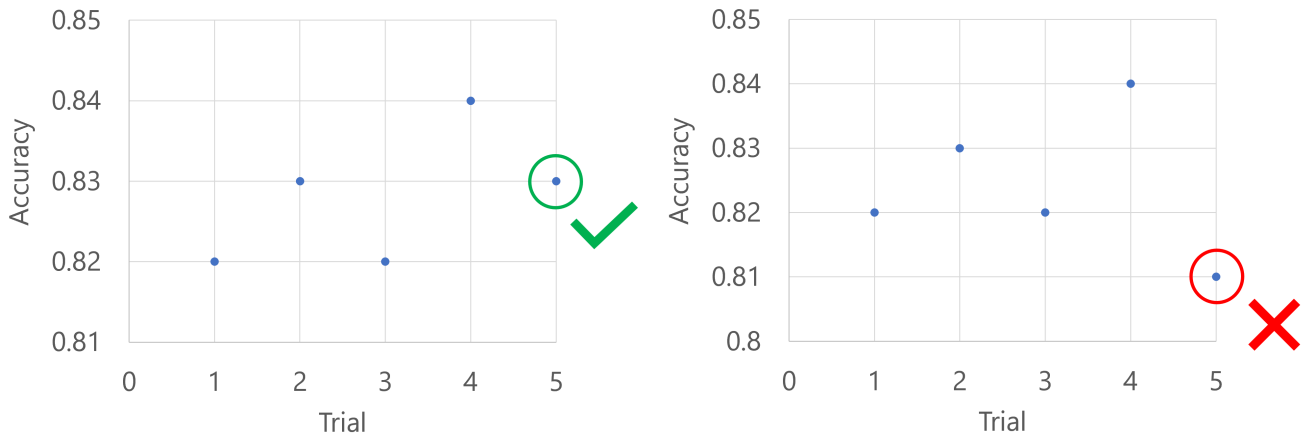
Truncation selection policy

A truncation selection policy cancels the lowest performing *X*% of trials at each evaluation interval based on the *truncation_percentage* value you specify for *X*.

For example, the following code applies a truncation selection policy with a delay of four trials, evaluates the policy at every interval, and uses a truncation percentage of 20%.

```
from azure.ai.ml.sweep import TruncationSelectionPolicy

sweep_job.early_termination = TruncationSelectionPolicy(
    evaluation_interval=1,
    truncation_percentage=20,
    delay_evaluation=4
)
```

Imagine the primary metric is the accuracy of the model. When the accuracy is logged for the fifth trial, the metric should **not** be in the worst 20% of the trials so far. In this case, 20% translates to one trial. In other words, if the fifth trial is **not** the worst performing model so far, the sweep job will continue. If the fifth trial has the lowest accuracy score of all trials so far, the sweep job will stop.



Use a sweep job for hyperparameter tuning

In Azure Machine Learning, you can tune hyperparameters by running a **sweep job**.

Create a training script for hyperparameter tuning

To run a sweep job, you need to create a training script just the way you would do for any other training job, except that your script ***must***:

- Include an argument for each hyperparameter you want to vary.
- Log the target performance metric with **MLflow**. A logged metric enables the sweep job to evaluate the performance of the trials it initiates, and identify the one that produces the best performing model.

> **Note**: Learn how to track machine learning experiments and models with MLflow within Azure Machine Learning.

For example, the following example script trains a logistic regression model using a `--regularization` argument to set the *regularization rate* hyperparameter, and logs the *accuracy* metric with the name `Accuracy`:

```
import argparse
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import mlflow

# get regularization hyperparameter
parser = argparse.ArgumentParser()
parser.add_argument('--regularization', type=float, dest='reg_rate', default=0.01)
args = parser.parse_args()
reg = args.reg_rate

# load the training dataset
data = pd.read_csv("data.csv")

# separate features and labels, and split for training/validatiom
X = data[['feature1','feature2','feature3','feature4']].values
y = data['label'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30)

# train a logistic regression model with the reg hyperparameter
model = LogisticRegression(C=1/reg, solver="liblinear").fit(X_train, y_train)

# calculate and log accuracy
y_hat = model.predict(X_test)
acc = np.average(y_hat == y_test)
mlflow.log_metric("Accuracy", acc)
```

Configure and run a sweep job

To prepare the sweep job, you must first create a base **command job** that specifies which script to run and defines the parameters used by the script:

```
from azure.ai.ml import command

# configure command job as base
job = command(
    code="./src",
    command="python train.py --regularization ${{inputs.reg_rate}}",
    inputs={
        "reg_rate": 0.01,
    },
    environment="AzureML-sklearn-0.24-ubuntu18.04-py37-cpu@latest",
    compute="aml-cluster",
    )
```

You can then override your input parameters with your search space:

```
from azure.ai.ml.sweep import Choice

command_job_for_sweep = job(
    reg_rate=Choice(values=[0.01, 0.1, 1]),
)
```

Finally, call `sweep()` on your command job to sweep over your search space:

```
from azure.ai.ml import MLClient

# apply the sweep parameter to obtain the sweep_job
sweep_job = command_job_for_sweep.sweep(
    compute="aml-cluster",
    sampling_algorithm="grid",
    primary_metric="Accuracy",
    goal="Maximize",
)

# set the name of the sweep job experiment
sweep_job.experiment_name="sweep-example"

# define the limits for this sweep
sweep_job.set_limits(max_total_trials=4, max_concurrent_trials=2, timeout=7200)

# submit the sweep
returned_sweep_job = ml_client.create_or_update(sweep_job)
```

Monitor and review sweep jobs

You can monitor sweep jobs in Azure Machine Learning studio. The sweep job will initiate trials for each hyperparameter combination to be tried. For each trial, you can review all logged metrics.

Additionally, you can evaluate and compare models by visualizing the trials in the studio. You can adjust each chart to show and compare the hyperparameter values and metrics for each trial.

> **Tip**: Learn more about how to visualize hyperparameter tuning jobs.

Perform hyperparameter tuning with a sweep job

Hyperparameters are variables that affect how a model is trained, but which can't be derived from the training data. Choosing the optimal hyperparameter values for model training can be difficult, and usually involved a great deal of trial and error.

In this exercise, you'll use Azure Machine Learning to tune hyperparameters by performing multiple training trials in parallel.

Before you start

You'll need an Azure subscription in which you have administrative-level access.

Provision an Azure Machine Learning workspace

An Azure Machine Learning *workspace* provides a central place for managing all resources and assets you need to train and manage your models. You can interact with the Azure Machine Learning workspace through the studio, Python SDK, and Azure CLI.

You'll use the Azure CLI to provision the workspace and necessary compute, and you'll use the Python SDK to run a command job.

Create the workspace and compute resources

To create the Azure Machine Learning workspace, a compute instance, and a compute cluster, you'll use the Azure CLI. All necessary commands are grouped in a Shell script for you to execute.

1. In a browser, open the Azure portal at `https://portal.azure.com/`, signing in with your Microsoft account.
2. Select the [>_] (*Cloud Shell*) button at the top of the page to the right of the search box. This opens a Cloud Shell pane at the bottom of the portal.
3. Select **Bash** if asked. The first time you open the cloud shell, you will be asked to choose the type of shell you want to use (*Bash* or *PowerShell*).
4. Check that the correct subscription is specified and that **No storage account required** is selected. Select **Apply**.
5. In the terminal, enter the following commands to clone this repo:

```
rm -r azure-ml-labs -f
git clone https://github.com/MicrosoftLearning/mslearn-azure-ml.git azure-ml-labs
```

> Use `SHIFT + INSERT` to paste your copied code into the Cloud Shell.

6. After the repo has been cloned, enter the following commands to change to the folder for this lab and run the **setup.sh** script it contains:

```
cd azure-ml-labs/Labs/09
./setup.sh
```

> Ignore any (error) messages that say that the extensions were not installed.

7. Wait for the script to complete - this typically takes around 5-10 minutes.

   Troubleshooting tip: Workspace creation error

   If you receive an error when running the setup script through the CLI, you need to provision the resources manually:

   1. In the Azure portal home page, select **+ Create a resource**.
   2. Search for *machine learning* and then select **Azure Machine Learning**. Select **Create**.
   3. Create a new Azure Machine Learning resource with the following settings:
      - **Subscription**: *Your Azure subscription*
      - **Resource group**: rg-dp100-labs
      - **Workspace name**: mlw-dp100-labs
      - **Region**: *Select the geographical region closest to you*
      - **Storage account**: *Note the default new storage account that will be created for your workspace*
      - **Key vault**: *Note the default new key vault that will be created for your workspace*
      - **Application insights**: *Note the default new application insights resource that will be created for your workspace*
      - **Container registry**: None (*one will be created automatically the first time you deploy a model to a container*)
   4. Select **Review + create** and wait for the workspace and its associated resources to be created - this typically takes around 5 minutes.
   5. Select **Go to resource** and in its **Overview** page, select **Launch studio**. Another tab will open in your browser to open the Azure Machine Learning studio.
   6. Close any pop-ups that appear in the studio.
   7. Within the Azure Machine Learning studio, navigate to the **Compute** page and select **+ New** under the **Compute instances** tab.
   8. Give the compute instance a unique name and then select **Standard_DS11_v2** as the virtual machine size.
   9. Select **Review + create** and then select **Create**.
   10. Next, select the **Compute clusters** tab and select **+ New**.

11. Choose the same region as the one where you created your workspace and then select **Standard_DS11_v2** as the virtual machine size. Select **Next**
12. Give the cluster a unique name and then select **Create**.

Clone the lab materials

When you've created the workspace and necessary compute resources, you can open the Azure Machine Learning studio and clone the lab materials into the workspace.

1. In the Azure portal, navigate to the Azure Machine Learning workspace named **mlw-dp100-...**.
2. Select the Azure Machine Learning workspace, and in its **Overview** page, select **Launch studio**. Another tab will open in your browser to open the Azure Machine Learning studio.
3. Close any pop-ups that appear in the studio.
4. Within the Azure Machine Learning studio, navigate to the **Compute** page and verify that the compute instance and cluster you created in the previous section exist. The compute instance should be running, the cluster should be idle and have 0 nodes running.
5. In the **Compute instances** tab, find your compute instance, and select the **Terminal** application.
6. In the terminal, install the Python SDK on the compute instance by running the following commands in the terminal:

   ```
   pip uninstall azure-ai-ml
   pip install azure-ai-ml
   ```

   > Ignore any (error) messages that say that the packages couldn't be found and uninstalled.

7. Run the following command to clone a Git repository containing notebooks, data, and other files to your workspace:

   ```
   git clone https://github.com/MicrosoftLearning/mslearn-azure-ml.git azure-ml-labs
   ```

8. When the command has completed, in the **Files** pane, click ↻ to refresh the view and verify that a new **Users/*your-user-name*/azure-ml-labs** folder has been created.

Tune hyperparameters with a sweep job

Now that you have all the necessary resources, you can run the notebook to submit a sweep job.

1. Open the **Labs/09/Hyperparameter tuning.ipynb** notebook.

   > Select **Authenticate** and follow the necessary steps if a notification appears asking you to authenticate.

2. Verify that the notebook uses the **Python 3.10 - AzureML** kernel.

3. Run all cells in the notebook.

Delete Azure resources

When you finish exploring Azure Machine Learning, you should delete the resources you've created to avoid unnecessary Azure costs.

1. Close the Azure Machine Learning studio tab and return to the Azure portal.
2. In the Azure portal, on the **Home** page, select **Resource groups**.
3. Select the **rg-dp100-...** resource group.
4. At the top of the **Overview** page for your resource group, select **Delete resource group**.
5. Enter the resource group name to confirm you want to delete it, and select **Delete**.

Run pipelines in Azure Machine Learning

Introduction

In Azure Machine Learning, you can experiment in notebooks and train (and retrain) machine learning models by running scripts as jobs.

In an enterprise data science process, you'll want to separate the overall process into individual tasks. You can group tasks together as **pipelines**. Pipelines are key to implementing an effective **Machine Learning Operations** (**MLOps**) solution in Azure.

> **Note**: The term *pipeline* is used extensively across various domains, including machine learning and software engineering. In Azure Machine Learning, a pipeline contains steps related to the training of a machine learning model. In Azure DevOps or GitHub, a pipeline can refer to a build or release pipelines, which perform the build and configuration tasks required to deliver software. In Azure Synapse Analytics, a pipeline is used to define the data ingestion and transformation process. The focus of this module is on Azure Machine Learning pipelines. However, bear in mind that it's possible to have pipelines across services interact with each other. For example, an Azure DevOps or Azure Synapse Analytics pipeline can trigger an Azure Machine Learning pipeline.

> **Tip**: Learn more about MLOps in relation to Azure Machine Learning with an introduction to machine learning operations

Create components

Components allow you to create reusable scripts that can easily be shared across users within the same Azure Machine Learning workspace. You can also use components to build an Azure Machine Learning pipeline.

Use a component

There are two main reasons why you'd use components:

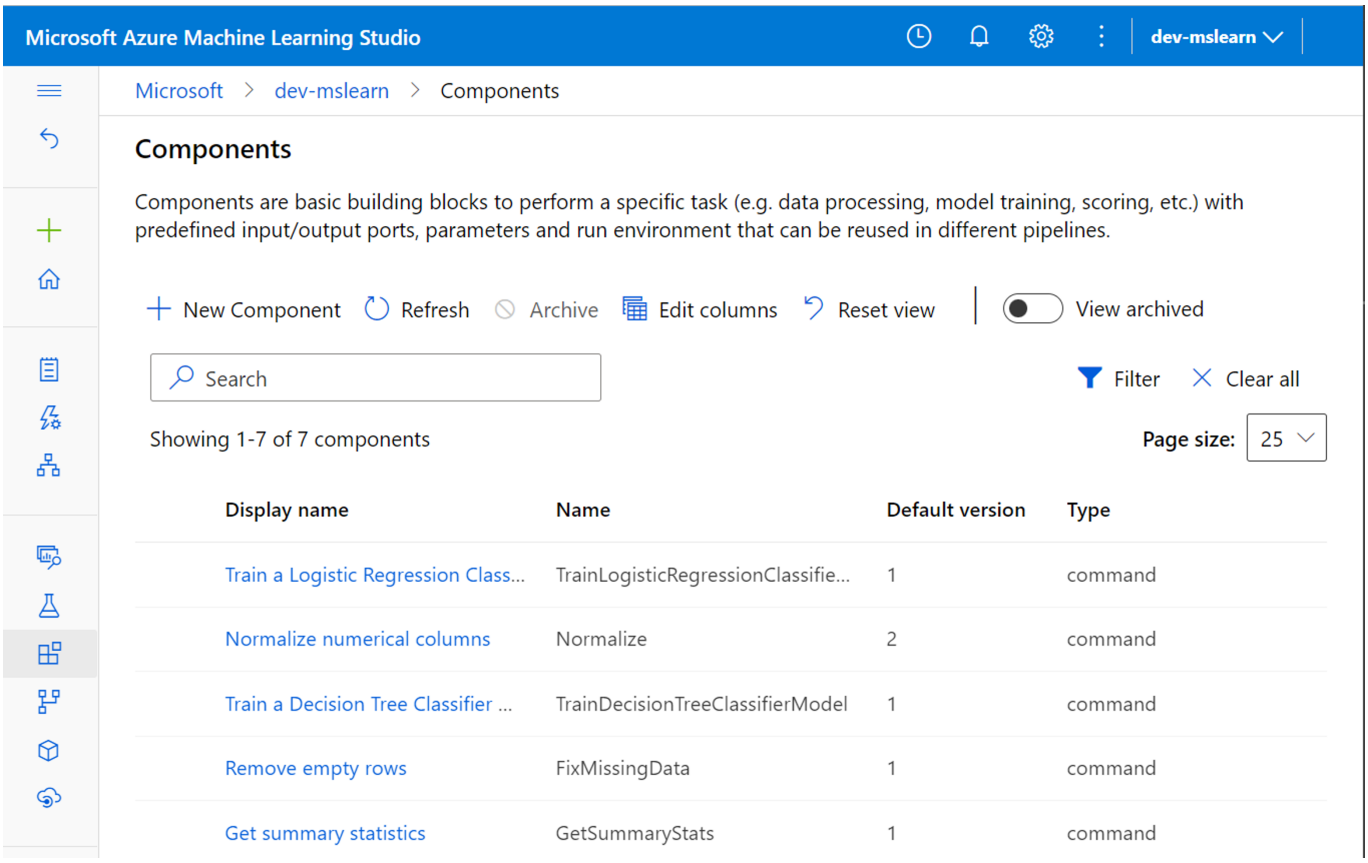
- To build a pipeline.
- To share ready-to-go code.

You'll want to create components when you're *preparing your code for scale*. When you're done with experimenting and developing, and ready to move your model to production.

Within Azure Machine Learning, you can create a component to store code (in your preferred language) within the workspace. Ideally, you design a component to perform a specific action that is relevant to your machine learning workflow.

For example, a component may consist of a Python script that normalizes your data, trains a machine learning model, or evaluates a model.

Components can be easily shared to other Azure Machine Learning users, who can reuse components in their own Azure Machine Learning pipelines.



Create a component

A component consists of three parts:

- **Metadata**: Includes the component's name, version, etc.
- **Interface**: Includes the expected input parameters (like a dataset or hyperparameter) and expected output (like metrics and artifacts).
- **Command, code and environment**: Specifies how to run the code.

To create a component, you need two files:

- A script that contains the workflow you want to execute.
- A YAML file to define the metadata, interface, and command, code, and environment of the component.

You can create the YAML file, or use the `command_component()` function as a decorator to create the YAML file.

> **Tip**: Here, we'll focus on creating a YAML file to create a component. Alternatively, learn more about how to create components using `command_component()`.

For example, you may have a Python script `prep.py` that prepares the data by removing missing values and normalizing the data:

```
# import libraries
import argparse
import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.preprocessing import MinMaxScaler

# setup arg parser
parser = argparse.ArgumentParser()

# add arguments
parser.add_argument("--input_data", dest='input_data',
                    type=str)
parser.add_argument("--output_data", dest='output_data',
                    type=str)

# parse args
args = parser.parse_args()

# read the data
df = pd.read_csv(args.input_data)

# remove missing values
df = df.dropna()

# normalize the data
scaler = MinMaxScaler()
num_cols = ['feature1','feature2','feature3','feature4']
df[num_cols] = scaler.fit_transform(df[num_cols])

# save the data as a csv
output_df = df.to_csv(
    (Path(args.output_data) / "prepped-data.csv"),
    index = False
)
```

To create a component for the `prep.py` script, you'll need a YAML file `prep.yml`:

```
$schema: https://azuremlschemas.azureedge.net/latest/commandComponent.schema.json
name: prep_data
display_name: Prepare training data
version: 1
type: command
inputs:
```

```
    input_data:
      type: uri_file
  outputs:
    output_data:
      type: uri_file
  code: ./src
  environment: azureml:AzureML-sklearn-0.24-ubuntu18.04-py37-cpu@latest
  command: >-
    python prep.py
    --input_data ${{inputs.input_data}}
    --output_data ${{outputs.output_data}}
```

Notice that the YAML file refers to the `prep.py` script, which is stored in the `src` folder. You can load the component with the following code:

```
from azure.ai.ml import load_component
parent_dir = ""

loaded_component_prep = load_component(source=parent_dir + "./prep.yml")
```

When you've loaded the component, you can use it in a pipeline or register the component.

Register a component

To use components in a pipeline, you'll need the script and the YAML file. To make the components accessible to other users in the workspace, you can also register components to the Azure Machine Learning workspace.

You can register a component with the following code:

```
prep = ml_client.components.create_or_update(prepare_data_component)
```

Create a pipeline

In Azure Machine Learning, a **pipeline** is a workflow of machine learning tasks in which each task is defined as a **component**.

Components can be arranged sequentially or in parallel, enabling you to build sophisticated flow logic to orchestrate machine learning operations. Each component can be run on a specific compute target, making it possible to combine different types of processing as required to achieve an overall goal.

A pipeline can be executed as a process by running the pipeline as a **pipeline job**. Each component is executed as a **child job** as part of the overall pipeline job.

Build a pipeline

An Azure Machine Learning pipeline is defined in a YAML file. The YAML file includes the pipeline job name, inputs, outputs, and settings.

You can create the YAML file, or use the `@pipeline()` function to create the YAML file.

> **Tip**: Review the reference documentation for the `@pipeline()` function.

For example, if you want to build a pipeline that first prepares the data, and then trains the model, you can use the following code:

```
from azure.ai.ml.dsl import pipeline

@pipeline()
def pipeline_function_name(pipeline_job_input):
    prep_data = loaded_component_prep(input_data=pipeline_job_input)
    train_model =
loaded_component_train(training_data=prep_data.outputs.output_data)

    return {
        "pipeline_job_transformed_data": prep_data.outputs.output_data,
        "pipeline_job_trained_model": train_model.outputs.model_output,
    }
```

To pass a registered data asset as the pipeline job input, you can call the function you created with the data asset as input:

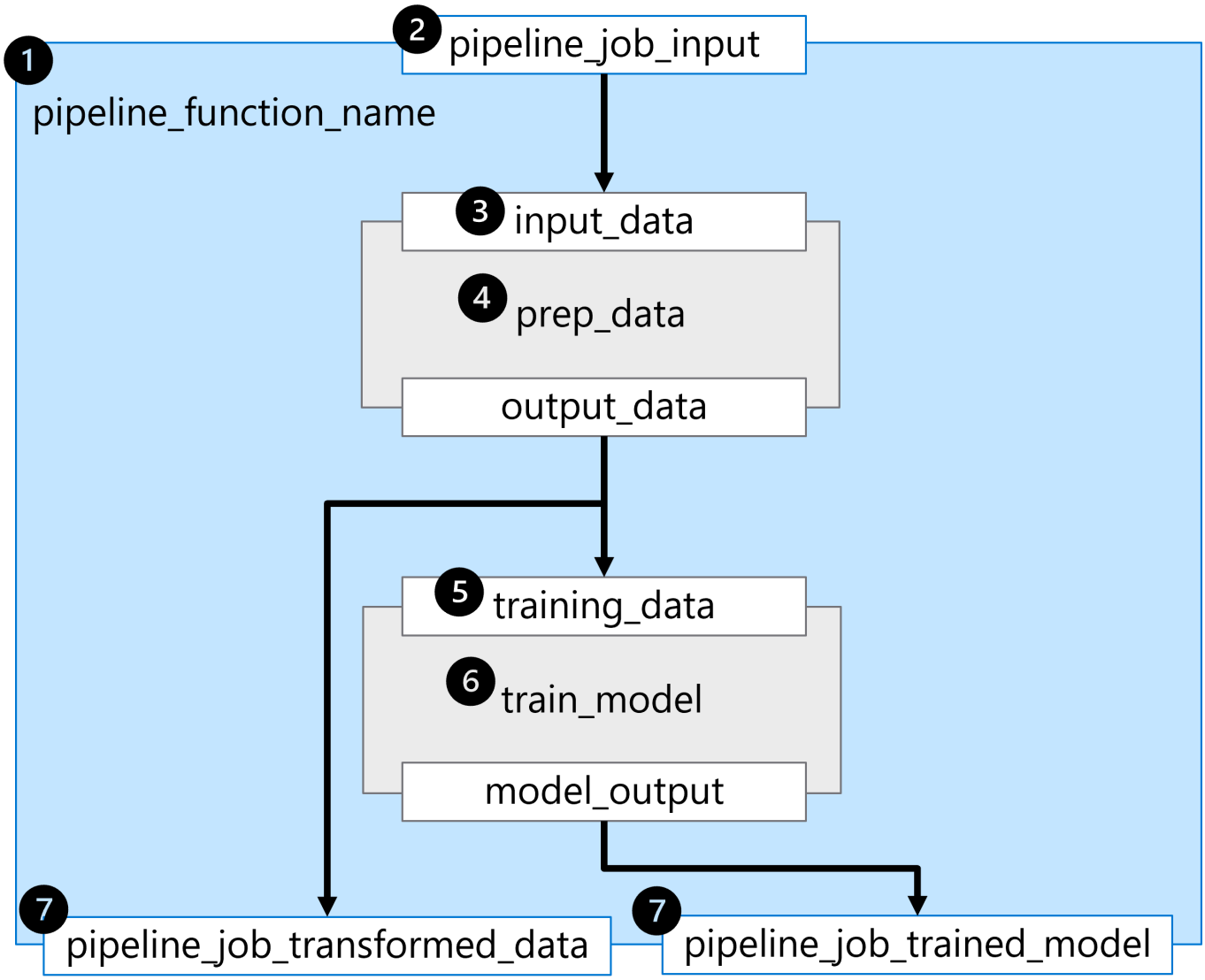
```
from azure.ai.ml import Input
from azure.ai.ml.constants import AssetTypes

pipeline_job = pipeline_function_name(
    Input(type=AssetTypes.URI_FILE,
    path="azureml:data:1"
))
```

The `@pipeline()` function builds a pipeline consisting of two sequential steps, represented by the two loaded components.

To understand the pipeline built in the example, let's explore it step by step:

1. The pipeline is built by defining the function `pipeline_function_name`.
2. The pipeline function expects `pipeline_job_input` as the overall pipeline input.
3. The first pipeline step requires a value for the input parameter `input_data`. The value for the input will be the value of `pipeline_job_input`.
4. The first pipeline step is defined by the loaded component for `prep_data`.
5. The value of the `output_data` of the first pipeline step is used for the expected input `training_data` of the second pipeline step.
6. The second pipeline step is defined by the loaded component for `train_model` and results in a trained model referred to by `model_output`.
7. Pipeline outputs are defined by returning variables from the pipeline function. There are two outputs:
    - `pipeline_job_transformed_data` with the value of `prep_data.outputs.output_data`

- `pipeline_job_trained_model` with the value of `train_model.outputs.model_output`



The result of running the `@pipeline()` function is a YAML file that you can review by printing the `pipeline_job` object you created when calling the function:

```
print(pipeline_job)
```

The output will be formatted as a YAML file, which includes the configuration of the pipeline and its components. Some parameters included in the YAML file are shown in the following example.

```yaml
display_name: pipeline_function_name
type: pipeline
inputs:
  pipeline_job_input:
    type: uri_file
    path: azureml:data:1
outputs:
  pipeline_job_transformed_data: null
  pipeline_job_trained_model: null
jobs:
```

```
    prep_data:
      type: command
      inputs:
        input_data:
          path: ${{parent.inputs.pipeline_job_input}}
      outputs:
        output_data: ${{parent.outputs.pipeline_job_transformed_data}}
    train_model:
      type: command
      inputs:
        input_data:
          path: ${{parent.outputs.pipeline_job_transformed_data}}
      outputs:
        output_model: ${{parent.outputs.pipeline_job_trained_model}}
  tags: {}
  properties: {}
  settings: {}
```

> **Tip**: Learn more about the pipeline job YAML schema to explore which parameters are included when building a component-based pipeline.

Run a pipeline job

When you've built a component-based pipeline in Azure Machine Learning, you can run the workflow as a **pipeline job**.

Configure a pipeline job

A pipeline is defined in a YAML file, which you can also create using the `@pipeline()` function. After you've used the function, you can edit the pipeline configurations by specifying which parameters you want to change and the new value.

For example, you may want to change the output mode for the pipeline job outputs:

```
# change the output mode
pipeline_job.outputs.pipeline_job_transformed_data.mode = "upload"
pipeline_job.outputs.pipeline_job_trained_model.mode = "upload"
```

Or, you may want to set the default pipeline compute. When a compute isn't specified for a component, it will use the default compute instead:

```
# set pipeline level compute
pipeline_job.settings.default_compute = "aml-cluster"
```

You may also want to change the default datastore to where all outputs will be stored:

```
# set pipeline level datastore
pipeline_job.settings.default_datastore = "workspaceblobstore"
```

To review your pipeline configuration, you can print the pipeline job object:

```
print(pipeline_job)
```

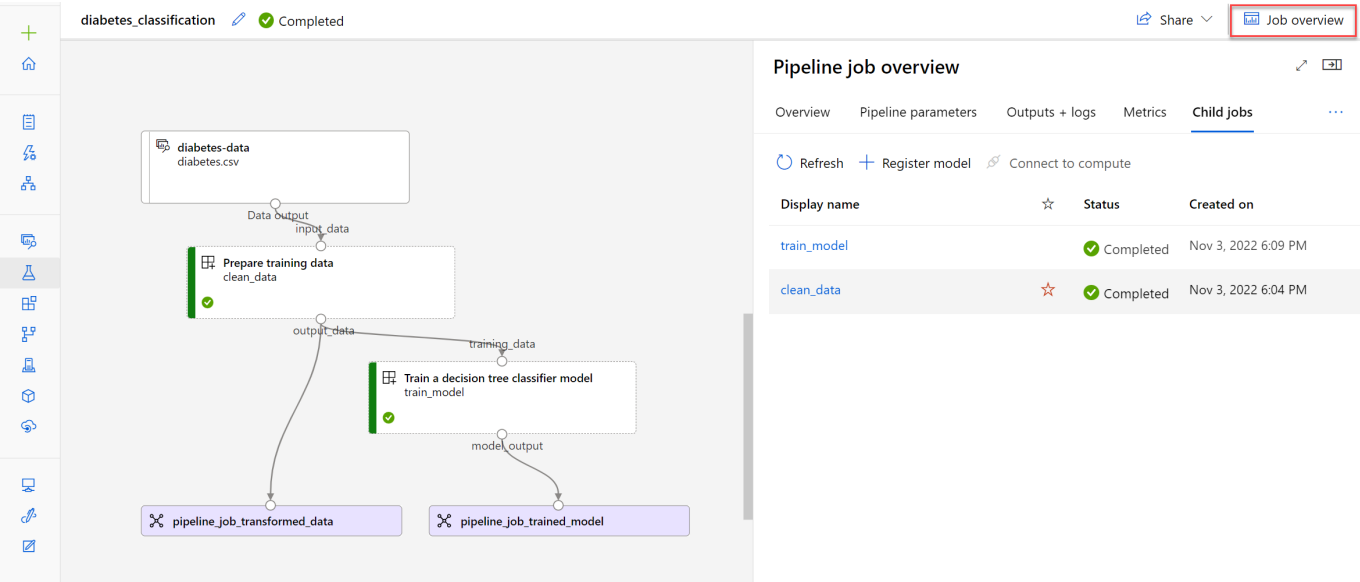
Run a pipeline job

When you've configured the pipeline, you're ready to run the workflow as a pipeline job.

To submit the pipeline job, run the following code:

```
# submit job to workspace
pipeline_job = ml_client.jobs.create_or_update(
    pipeline_job, experiment_name="pipeline_job"
)
```

After you submit a pipeline job, a new job will be created in the Azure Machine Learning workspace. A pipeline job also contains child jobs, which represent the execution of the individual components. The Azure Machine Learning studio creates a graphical representation of your pipeline. You can expand the **Job overview** to explore the pipeline parameters, outputs, and child jobs:



To troubleshoot a failed pipeline, you can check the outputs and logs of the pipeline job and its child jobs.

- If there's an issue with the configuration of the pipeline itself, you'll find more information in the outputs and logs of the pipeline job.
- If there's an issue with the configuration of a component, you'll find more information in the outputs and logs of the child job of the failed component.

Schedule a pipeline job

A pipeline is ideal if you want to get your model ready for production. Pipelines are especially useful for automating the retraining of a machine learning model. To automate the retraining of a model, you can schedule a pipeline.

To schedule a pipeline job, you'll use the `JobSchedule` class to associate a schedule to a pipeline job.

There are various ways to create a schedule. A simple approach is to create a time-based schedule using the `RecurrenceTrigger` class with the following parameters:

- `frequency`: Unit of time to describe how often the schedule fires. Value can be either `minute`, `hour`, `day`, `week`, or `month`.
- `interval`: Number of frequency units to describe how often the schedule fires. Value needs to be an integer.

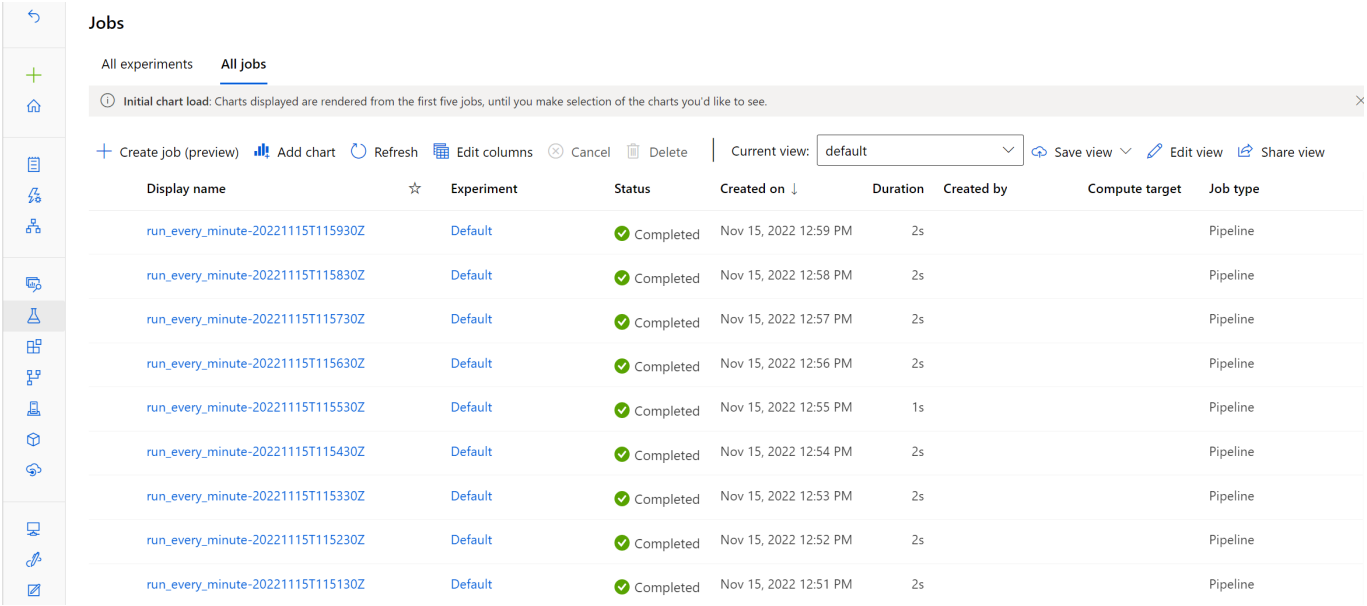
To create a schedule that fires every minute, run the following code:

```
from azure.ai.ml.entities import RecurrenceTrigger

schedule_name = "run_every_minute"

recurrence_trigger = RecurrenceTrigger(
    frequency="minute",
    interval=1,
)
```

To schedule a pipeline, you'll need `pipeline_job` to represent the pipeline you've built:

```
from azure.ai.ml.entities import JobSchedule

job_schedule = JobSchedule(
    name=schedule_name, trigger=recurrence_trigger, create_job=pipeline_job
)

job_schedule = ml_client.schedules.begin_create_or_update(
    schedule=job_schedule
).result()
```

The display names of the jobs triggered by the schedule will be prefixed with the name of your schedule. You can review the jobs in the Azure Machine Learning studio:



To delete a schedule, you first need to disable it:

```
ml_client.schedules.begin_disable(name=schedule_name).result()
ml_client.schedules.begin_delete(name=schedule_name).result()
```

> **Tip**: Learn more about the schedules you can create to trigger pipeline jobs in Azure Machine Learning. Or, explore an example notebook to learn how to work with schedules.

Run pipelines in Azure Machine Learning

You can use the Python SDK to perform all of the tasks required to create and operate a machine learning solution in Azure. Rather than perform these tasks individually, you can use pipelines to orchestrate the steps required to prepare data, run training scripts, and other tasks.

In this exercise, you'll run multiple scripts as a pipeline job.

Before you start

You'll need an Azure subscription in which you have administrative-level access.

Provision an Azure Machine Learning workspace

An Azure Machine Learning *workspace* provides a central place for managing all resources and assets you need to train and manage your models. You can interact with the Azure Machine Learning workspace through the studio, Python SDK, and Azure CLI.

You'll use the Azure CLI to provision the workspace and necessary compute, and you'll use the Python SDK to run a command job.

Create the workspace and compute resources

To create the Azure Machine Learning workspace, a compute instance, and a compute cluster, you'll use the Azure CLI. All necessary commands are grouped in a Shell script for you to execute.

1. In a browser, open the Azure portal at `https://portal.azure.com/`, signing in with your Microsoft account.

2. Select the [>_] (*Cloud Shell*) button at the top of the page to the right of the search box. This opens a Cloud Shell pane at the bottom of the portal.

3. Select **Bash** if asked. The first time you open the cloud shell, you will be asked to choose the type of shell you want to use (*Bash* or *PowerShell*).

4. Check that the correct subscription is specified and that **No storage account required** is selected. Select **Apply**.

5. In the terminal, enter the following commands to clone this repo:

```
rm -r azure-ml-labs -f
git clone https://github.com/MicrosoftLearning/mslearn-azure-ml.git azure-ml-labs
```

> Use `SHIFT + INSERT` to paste your copied code into the Cloud Shell.

6. After the repo has been cloned, enter the following commands to change to the folder for this lab and run the **setup.sh** script it contains:

```
cd azure-ml-labs/Labs/09
./setup.sh
```

> Ignore any (error) messages that say that the extensions were not installed.

7. Wait for the script to complete - this typically takes around 5-10 minutes.

   Troubleshooting tip: Workspace creation error

   If you receive an error when running the setup script through the CLI, you need to provision the resources manually:

   1. In the Azure portal home page, select **+ Create a resource**.
   2. Search for *machine learning* and then select **Azure Machine Learning**. Select **Create**.
   3. Create a new Azure Machine Learning resource with the following settings:
      - **Subscription**: *Your Azure subscription*
      - **Resource group**: rg-dp100-labs
      - **Workspace name**: mlw-dp100-labs
      - **Region**: *Select the geographical region closest to you*
      - **Storage account**: *Note the default new storage account that will be created for your workspace*
      - **Key vault**: *Note the default new key vault that will be created for your workspace*
      - **Application insights**: *Note the default new application insights resource that will be created for your workspace*

    - **Container registry**: None (*one will be created automatically the first time you deploy a model to a container*)
4. Select **Review + create** and wait for the workspace and its associated resources to be created - this typically takes around 5 minutes.
5. Select **Go to resource** and in its **Overview** page, select **Launch studio**. Another tab will open in your browser to open the Azure Machine Learning studio.
6. Close any pop-ups that appear in the studio.
7. Within the Azure Machine Learning studio, navigate to the **Compute** page and select **+ New** under the **Compute instances** tab.
8. Give the compute instance a unique name and then select **Standard_DS11_v2** as the virtual machine size.
9. Select **Review + create** and then select **Create**.
10. Next, select the **Compute clusters** tab and select **+ New**.
11. Choose the same region as the one where you created your workspace and then select **Standard_DS11_v2** as the virtual machine size. Select **Next**.
12. Give the cluster a unique name and then select **Create**.
13. Download the training data from https://github.com/MicrosoftLearning/mslearn-azure-ml/raw/refs/heads/main/Labs/09/data/diabetes.csv
14. In the Azure Machine Learning studio, navigate to the **Data** page and select **+ Create**.
15. Name the data asset **diabetes-data** and verify that the type **File (uri_file)** is selected. Select **Next**.
16. Select **From local files** as your data source and then select **Next**.
17. Verify that **Azure Blob Storage** and **workspaceblobstore** are selected as your destination storage type and datastore respectively. Select **Next**.
18. Upload the .csv file you downloaded previously and then select **Next**.
19. Review the settings for your data asset and then select **Create**.

Clone the lab materials

When you've created the workspace and necessary compute resources, you can open the Azure Machine Learning studio and clone the lab materials into the workspace.

1. In the Azure portal, navigate to the Azure Machine Learning workspace named **mlw-dp100-...**.

2. Select the Azure Machine Learning workspace, and in its **Overview** page, select **Launch studio**. Another tab will open in your browser to open the Azure Machine Learning studio.

3. Close any pop-ups that appear in the studio.

4. Within the Azure Machine Learning studio, navigate to the **Compute** page and verify that the compute instance and cluster you created in the previous section exist. The compute instance should be running, the cluster should be idle and have 0 nodes running.

5. In the **Compute instances** tab, find your compute instance, and select the **Terminal** application.

6. In the terminal, install the Python SDK on the compute instance by running the following commands in the terminal:

```
pip uninstall azure-ai-ml
pip install azure-ai-ml
```

> Ignore any (error) messages that say that the packages couldn't be found and uninstalled.

7. Run the following command to clone a Git repository containing notebooks, data, and other files to your workspace:

```
git clone https://github.com/MicrosoftLearning/mslearn-azure-ml.git azure-ml-labs
```

8. When the command has completed, in the **Files** pane, click ↻ to refresh the view and verify that a new **Users/*your-user-name*/azure-ml-labs** folder has been created.

Run scripts as a pipeline job

The code to build and submit a pipeline with the Python SDK is provided in a notebook.

1. Open the **Labs/09/Run a pipeline job.ipynb** notebook.

   > Select **Authenticate** and follow the necessary steps if a notification appears asking you to authenticate.

2. Verify that the notebook uses the **Python 3.10 - AzureML** kernel.

3. Run all cells in the notebook.

Delete Azure resources

When you finish exploring Azure Machine Learning, you should delete the resources you've created to avoid unnecessary Azure costs.

1. Close the Azure Machine Learning studio tab and return to the Azure portal.
2. In the Azure portal, on the **Home** page, select **Resource groups**.
3. Select the **rg-dp100-...** resource group.
4. At the top of the **Overview** page for your resource group, select **Delete resource group**.
5. Enter the resource group name to confirm you want to delete it, and select **Delete**.

Trigger Azure Machine Learning jobs with GitHub Actions

Introduction

Imagine you're a machine learning engineer, working together with a data science team on a diabetes classification model. The workflow created by the data science team preprocesses data and trains the model. You want to automatically execute the workflow. By doing so, you'll enable automated training (and retraining) of the classification model in different environments, driven by different events.

Automation is an important part in machine learning operations (MLOps). Similar to DevOps, MLOps allows for rapid development and delivery of machine learning artifacts to consumers of those artifacts. An effective MLOps strategy allows for the creation of automated workflows to train, test, and deploy machine learning models while also ensuring model quality is maintained.

Using GitHub Actions, you'll automatically execute an Azure Machine Learning job to train a model. To execute your Azure Machine Learning jobs with GitHub Actions, you'll save your Azure credentials as a secret in GitHub. You'll then define the GitHub Action using YAML.

Understand the business problem

You work at Proseware, a young start-up, aiming to improve health care. Together with the data science team, you've recently finished work on operationalizing a diabetes classification model. In other words, you've converted notebooks to scripts that you can execute as an Azure Machine Learning job.

During a presentation of the end-to-end solution to the business and technical stakeholders at Proseware, several questions came up around how to scale the use of this model both from a model creation standpoint and from a consumption standpoint.

In health care, many models use medical data of patients to predict diseases. From previous projects, we've learned that these models are often highly dependent on the geographical location of the population the model is trained on. To make this model scalable, we need to ensure that different versions of the model can automatically be trained based on different data segments.

In the meeting, the business and technical stakeholders have decided to implement a **machine learning operations (MLOps)** strategy to allow for the rapid creation, update, and deployment of models such as the classification model the data science team has developed for the practitioner web app.

As Proseware uses GitHub to version control its code, the decision was made to use **GitHub Actions** as the automation component of the MLOps strategy.

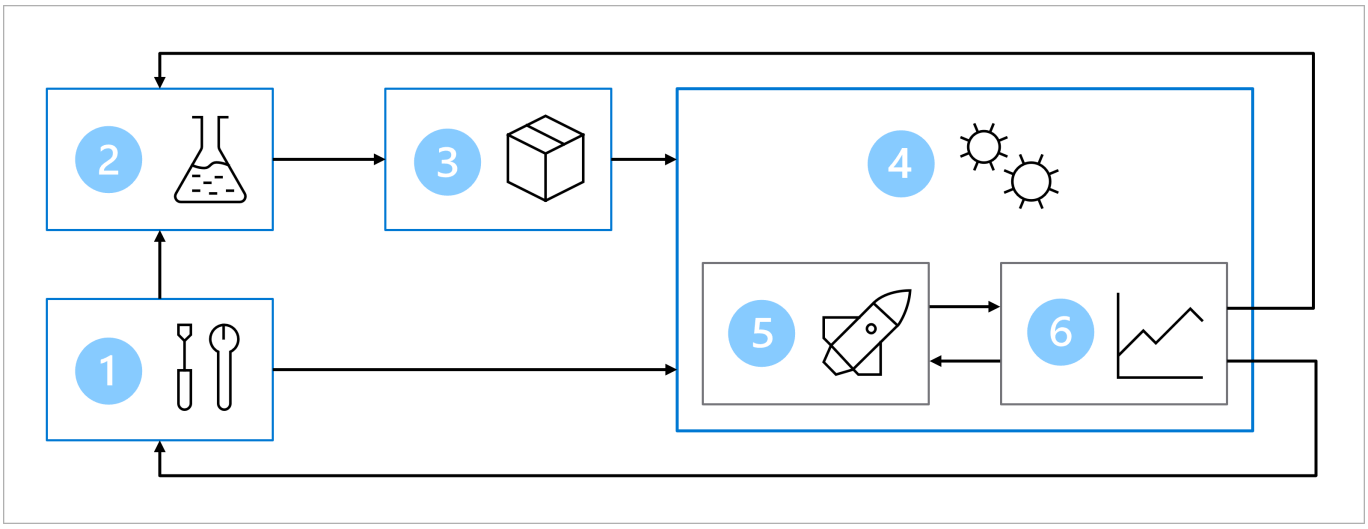
The first step in implementing the automation process is to develop a GitHub Action to train the diabetes classification model using Azure Machine Learning jobs.

To create the GitHub Action to trigger model training using Azure Machine Learning compute, you'll want to:

- Create a **service principal** using the Azure CLI.
- Store the credentials of the service principal as a **secret** in GitHub.
- Create a GitHub Action to train the model using Azure Machine Learning compute.

Explore the solution architecture

It's important to understand the overall picture before moving ahead with the implementation to ensure all the requirements are met. We also want to ensure the approach is easily adaptable in the future. The focus of this exercise is to start to use GitHub Actions as the orchestration and automation tool for the machine learning operations (MLOps) strategy defined in the solution architecture.



> **Note**: The diagram is a simplified representation of a MLOps architecture. To view a more detailed architecture, explore the various use cases in the MLOps (v2) solution accelerator.

The architecture includes:

1. **Setup**: Create all necessary Azure resources for the solution.
2. **Model development (inner loop)**: Explore and process the data to train and evaluate the model.
3. **Continuous integration**: Package and register the model.
4. **Model deployment (outer loop)**: Deploy the model.
5. **Continuous deployment**: Test the model and promote to production environment.
6. **Monitoring**: Monitor model and endpoint performance.

Specifically, we're going to be automating the training portion of the model development, or inner loop, which will ultimately allow us to quickly train and register multiple models for deployment to staging and production environments.

The Azure Machine Learning workspace, Azure Machine Learning compute, and GitHub repository have all been created for you by the infrastructure team.

In addition, the code to train the classification model is production-ready and the data needed to train the model is available in an Azure Blob Storage connected to the Azure Machine Learning workspace.

Your implementation will enable the move from inner to outer loop to be an automated process that happens whenever a data scientist pushes new model code to the GitHub repository, enabling the continuous delivery of machine learning models to downstream consumers of the model, like the web application that will use the diabetes classification model.

Use GitHub Actions for model training

GitHub Actions is a platform that allows you to automate tasks triggered by events that occur within a GitHub repository. A GitHub Actions workflow consist of **jobs**. A job groups a **set of steps** that you can define. One of these steps can use the CLI (v2) to run an **Azure Machine Learning job** to train a model.

To automate model training with GitHub Actions, you'll need to:

- Create a service principal using the Azure CLI.
- Store the Azure credentials in a GitHub secret.

- Define a GitHub Action in YAML.

Create a service principal

When you use GitHub Actions to automate Azure Machine Learning jobs, you need to use a service principal to authenticate GitHub to manage the Azure Machine Learning workspace. For example, to train a model using Azure Machine Learning compute, you or any tool that you use, needs to be authorized to use that compute.

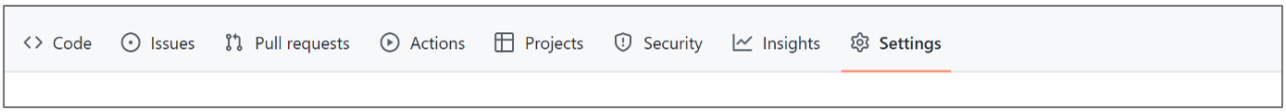
> **Tip**: Learn more about how to use GitHub Actions to connect to Azure

Store the Azure credentials

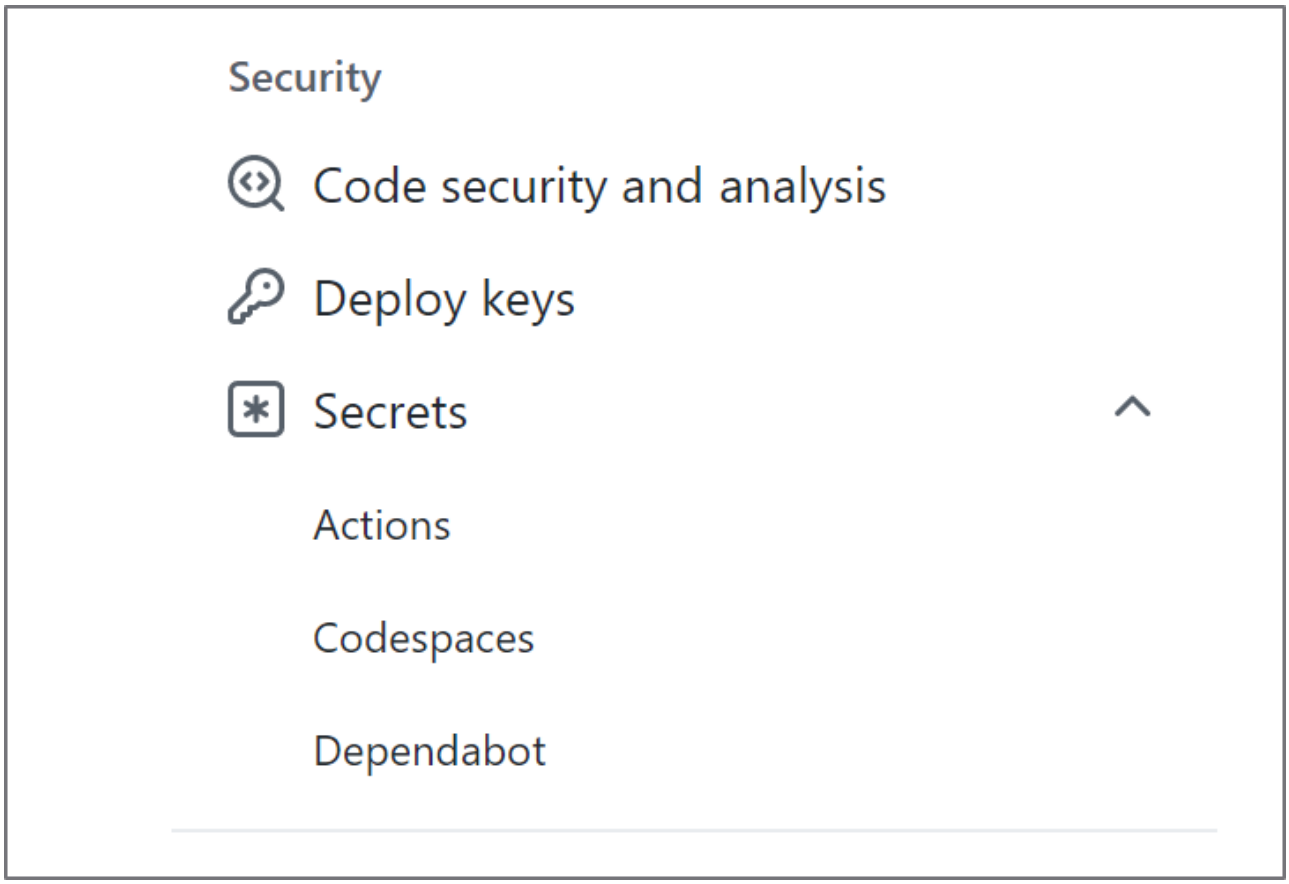
The Azure credentials you need to authenticate should not be stored in your code or plain text and should instead be stored in a GitHub secret.

To add a secret to your GitHub repository:

1. Navigate to the **Settings** tab.



2. In the **Settings** tab, under **Security**, expand the **Secrets** option and select **Actions**.



3. Enter your Azure credentials as a secret and name the secret `AZURE_CREDENTIALS`.

4. To use a secret containing Azure credentials in a GitHub Action, refer to the secret in the YAML file.

```yaml
on: [push]

name: Azure Login Sample

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Log in with Azure
        uses: azure/login@v1
        with:
          creds: '${{secrets.AZURE_CREDENTIALS}}'
```

Define the GitHub Action

To define a workflow, you'll need to create a YAML file. You can trigger the workflow to train a model manually or with a push event. Manually triggering the workflow is ideal for testing, while automating it with an event is better for automation.

To configure a GitHub Actions workflow so that you can trigger it manually, use `on: workflow_dispatch`. To trigger a workflow with a push event, use `on: [push]`.

Once the GitHub Actions workflow is triggered, you can add various steps to a job. For example, you can use a step to run an Azure Machine Learning job:

```yaml
name: Manually trigger an Azure Machine Learning job

on:
  workflow_dispatch:

jobs:
  train-model:
    runs-on: ubuntu-latest
    steps:
    - name: Trigger Azure Machine Learning job
      run: |
        az ml job create --file src/job.yml
```

> **Tip**: Learn more about GitHub Actions, including core concepts and essential terminology..

Trigger GitHub Actions with feature-based development

Introduction

Automation is one of the most important practices of **machine learning operations** (**MLOps**). By automating tasks, you can deploy new models to production more quickly.

Next to automation, another key aspect of MLOps is **source control** to manage code and track any changes.

Together, you can use automation and source control to trigger tasks in the machine learning workflow based on changes to the code. However, you want the automated task to be triggered only when the code changes have been verified and approved.

For example, after retraining a model using new hyperparameter values, you want to update the hyperparameter in the source code. After verifying and approving the change to the code that is used to train the model, you want to trigger the new model to be trained.

GitHub is a platform that offers GitHub Actions for automation and repositories using Git for source control. You can configure your GitHub Actions workflows to be triggered by a change in your repo.

Understand the business problem

As a machine learning engineer at Proseware, you collaborate with many technical stakeholders. Next to working with the data science team who trained a diabetes classification model, you also work together with the software developers responsible of the web application (used by practitioners) that will consume the model.

To adapt to new requirements, the web app will be updated over time, and similarly, the model is also expected to change over time. Whenever there's *data drift* or a decrease in *model performance*, the data science team will be asked to revise the model and update the code accordingly.

Whenever a change to the model is needed, the data science team will need to experiment, test, and package the model. While they're working on a new and improved model, the model in production should remain unchanged to ensure a stable experience for the practitioners working with the web app.

As a machine learning engineer, you want to set up **feature-based development** for the data scientists. By working with branches in your source control, you'll protect the main branch, which contains the production code, and you'll allow data scientists to safely experiment in their own branches.

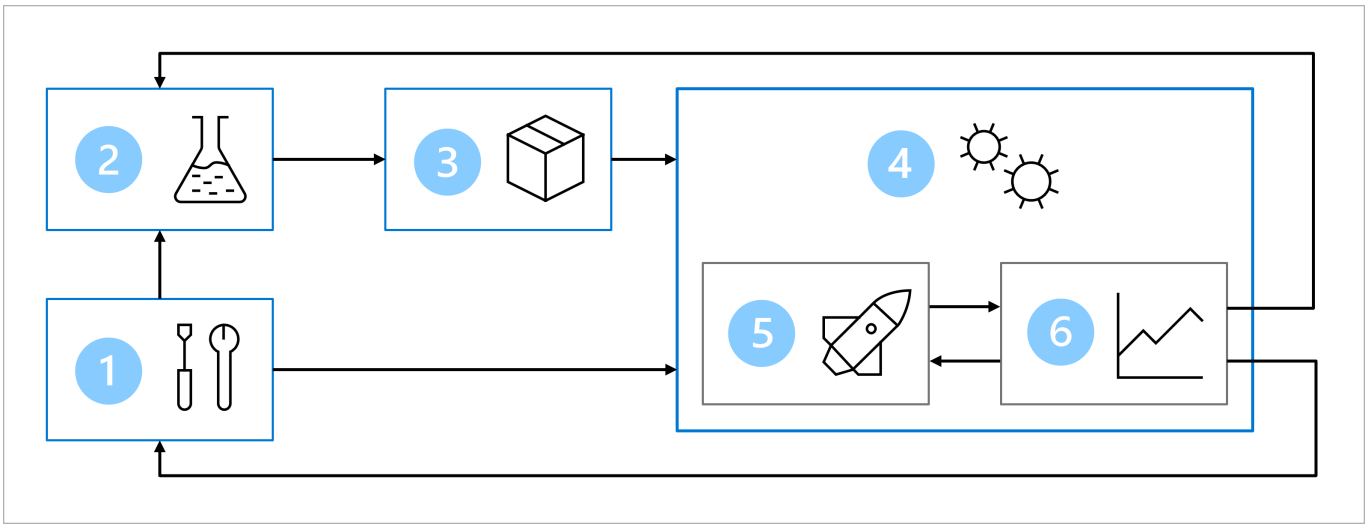
To set up feature-based development, you'll want to:

- **Block** any direct pushes to the main branch.
- Work with **pull requests** whenever an update to the code is needed.
- Trigger code quality checks whenever a pull request is created to **automatically verify the code**.
- Merge a pull request only when changes are **approved manually**.

Explore the solution architecture

Let's revise the machine learning operations (MLOps) architecture to understand the purpose of what we're trying to achieve.

Imagine that together with the data science and software development team, you've agreed on the following architecture to train, test, and deploy the diabetes classification model:



> **Note**: The diagram is a simplified representation of a MLOps architecture. To view a more detailed architecture, explore the various use cases in the MLOps (v2) solution accelerator.

The architecture includes:

1. **Setup**: Create all necessary Azure resources for the solution.
2. **Model development (inner loop)**: Explore and process the data to train and evaluate the model.
3. **Continuous integration**: Package and register the model.
4. **Model deployment (outer loop)**: Deploy the model.
5. **Continuous deployment**: Test the model and promote to production environment.
6. **Monitoring**: Monitor model and endpoint performance.

The data science team is responsible for the model development. The software development team is responsible for integrating the deployed model with the web app used by practitioners to assess whether a patient has diabetes. You're responsible of taking the model from model development to model deployment.

You expect the data science team to constantly propose changes to the scripts used to train the model. Whenever there's a change to the training script, you need to retrain the model and redeploy the model to the existing endpoint.

You want to allow the data science team to experiment, without touching the code ready for production. You also want to ensure that any new or updated code automatically goes through agreed upon quality checks. After you verify the code to train the model, you'll use the updated training script to train a new model and deploy it.

To keep track of changes and to verify your code before updating the production code, it's *necessary* to work with branches. You've agreed with the data science team that every time they want to make a change, they'll create a **feature branch** to create a copy of the code and make their changes to the copy.

Any data scientist can create a feature branch and work in there. Once they've updated the code and want that code to be the new production code, they'll have to create a **pull request**. In the pull request, it will be visible for others what the proposed changes are, giving others the opportunity to review and discuss the changes.

Whenever a pull request is created, you want to automatically check whether the code works and that the quality of the code is up to your organization's standards. After the code passes the quality checks, the lead

data scientist needs to review the changes and approve the updates before the pull request can be merged, and the code in the main branch can be updated accordingly.

> **Important**: **No one** should ever be allowed to **push changes to the main branch**. To safeguard your code, especially production code, you'll want to enforce that the main branch can only be updated through pull requests that need to be approved.

Trigger a workflow

No one should be allowed to push any changes directly to the main branch in your code repository. Ideally, if any development is necessary, you should make changes to a copy of the code in a **branch**.

A common approach is to work with **feature branches**, where a branch is used to work on a feature. For example, the data science team may need to improve the model performance and will try to do so by experimenting with hyperparameter values. The team can create a branch, update the hyperparameter value in the training script. And once done with experimenting, a data scientist can create a **pull request** to **merge** the branch with the main repo.

Working with branches and pull requests allows you to verify any changes to your code before merging them with the main branch. Pull requests can also be used as a trigger for GitHub Actions to automate tasks that need to follow a proposed update to the code, like automatic code quality checks.

To use feature-based development together with automation, you'll need to:

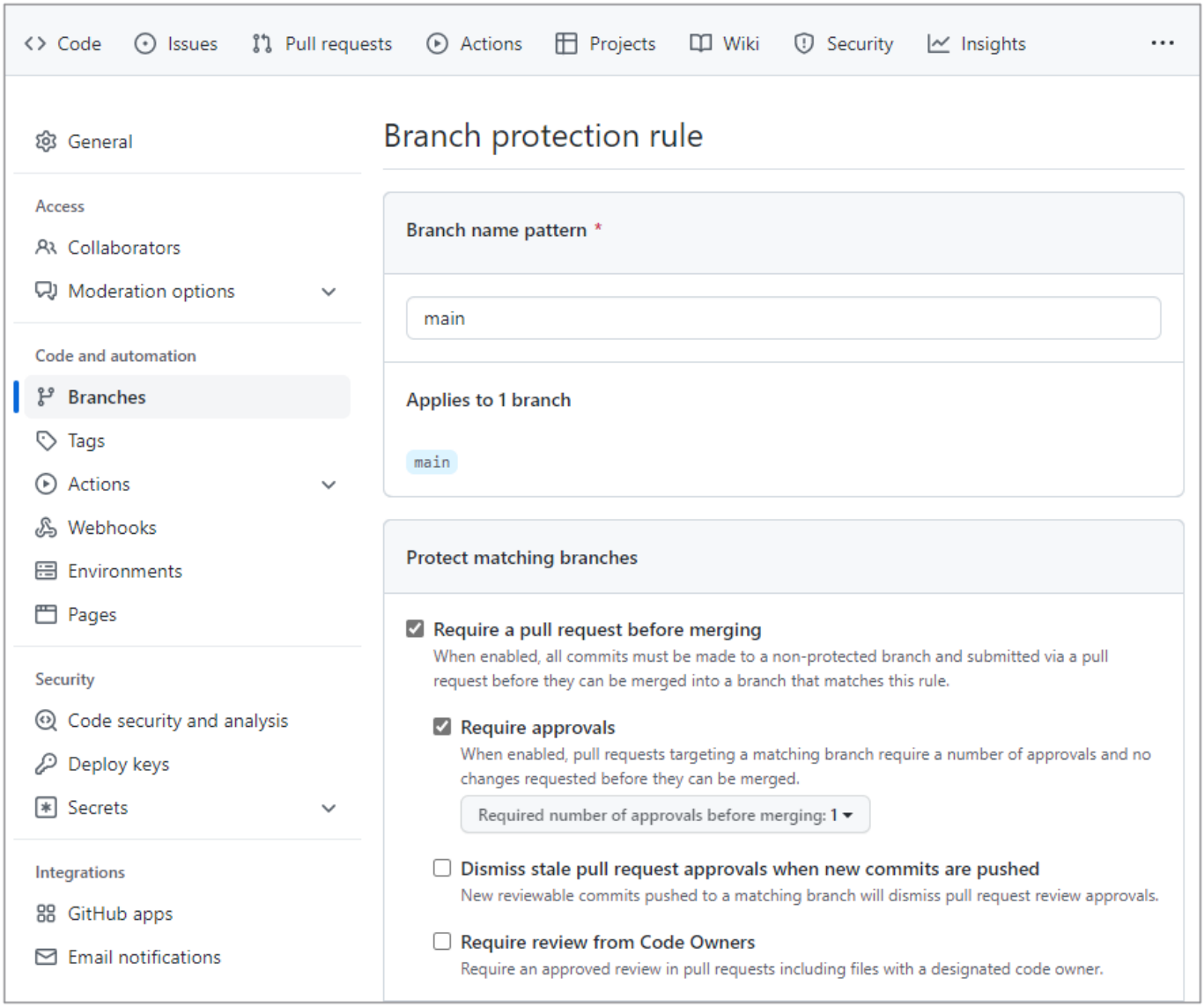
- Create a branch protection rule to block direct pushes to main.
- Create a branch to update the code.
- Trigger a GitHub Actions workflow when opening a pull request.

Create a branch protection rule

To protect your code, you want to **block any direct pushes to the main branch**. Blocking direct pushes means that no one will be allowed to directly push any code changes to the main branch. Instead, changes to the main branch can be made by merging pull requests.

To protect the main branch, enable a **branch protection rule** in GitHub:

1. Navigate to the **Settings** tab in your repo.
2. In the **Settings** tab, under **Code and automation**, select **Branches**.
3. Select **Add rule**.
4. Enter `main` under **Branch name pattern**.
5. Enable **Require a pull request before merging** and **Require approvals**.
6. Save your changes.



Create a branch to update the code.

Whenever you want to edit the code, you'll have to create a branch and work in there. Once you want to make your changes final, you can create a pull request to merge the feature branch with the main branch.

> **Tip**: Learn more about source control for machine learning projects and working with feature-based development.

Trigger a GitHub Actions workflow

Finally, you may want to use the creation of pull requests as a trigger for GitHub Actions workflows. For example, whenever someone makes changes to the code, you'll want to run some code quality checks.

Only when the edited code has passed the quality checks and someone has verified the proposed changes, do you want to actually merge the pull request.

To trigger a GitHub Actions workflow, you can use `on: [pull_request]`. When you use this trigger, your workflow will run whenever the pull request is created.

If you want a workflow to run whenever a pull request is merged, you'll need to use another trigger. Merging a pull request is essentially a push to the main branch. So, to trigger a workflow to run when a pull request is

merged, use the following trigger in the GitHub Actions workflow:

```
on:
  push:
    branches:
      - main
```

Work with environments in GitHub Actions

Introduction

Imagine you're a machine learning engineer, tasked with taking a model from development to production. To train, test, and deploy a machine learning model it's best to use **environments** as part of your **machine learning operations** (**MLOps**) strategy.

After a data scientist has trained and tested the model, you'll want to deploy the model, test the deployment, and finally deploy the model to production where it will be consumed at a large scale. In line with software development practices, these tasks should be performed in different environments. By using environments like a development, staging, and production environment, you can separate the MLOps workflow.

To create different environments, you can create different Azure Machine Learning workspaces that are linked to separate GitHub environments. By using GitHub Actions, you can automate workflows across environments, adding gated approvals to mitigate risks.

Understand the business problem

Imagine you're a machine learning engineer at Proseware, a young start-up working on a new health care app. The diabetes classification model, created by the data scientists is the first model to be integrated with the app. After you talk to the larger team, it turns out that the goal is to have multiple models integrated with the web app.

When the diabetes classification model proves to be successful, Proseware wants to add more machine learning models, so that practitioners can faster diagnose patients for various diseases. For every new model, the data science team will need to be able to experiment in a safe environment. Once the new model is accurate enough to be integrated with the web app, it should be tested before deploying it to an endpoint that will be called from the web app.

Together with the team, you decide it's best to use different environments:

- **Development** for experimentation.
- **Staging** for testing.
- **Production** for deploying the model to the production endpoint.

For each environment, you'll create a separate Azure Machine Learning workspace. By keeping the workspaces separate for each environment, you'll be able to protect data and resources. For example, the development workspace won't contain any personal data from patients. And the data scientists will only have access to the development workspace, as they only need an environment for experimentation, and don't need access to any of the production code or resources.

As a machine learning engineer, you do need to ensure that whatever the data scientists build, will be easily moved across environments. Once a new model is ready to be deployed, you want the model to be trained and tested in the staging environment. After testing the code, the model, and the deployment, you want to deploy the model in the production environment. Parts of this process can be automated to speed up the process.

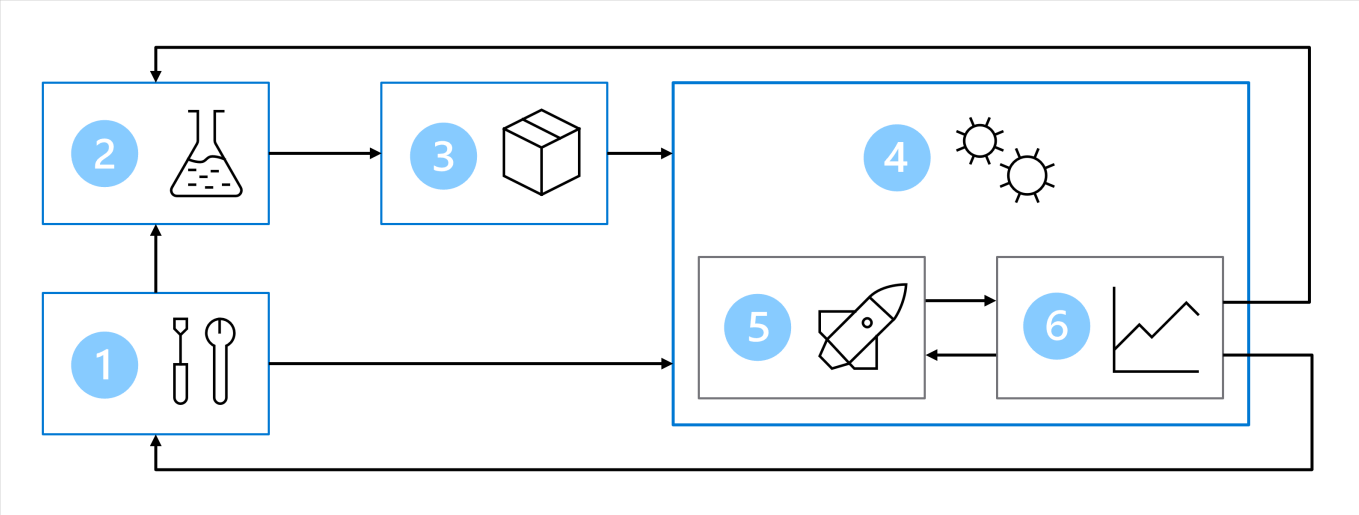
To work with environments, you'll want to:

- Create **environments** in your GitHub repository.
- Store credentials to each Azure Machine Learning workspace as an environment **secret** in GitHub.
- Add **required reviewers** to environments for **gated approval**.
- Use environments in your GitHub Actions workflows.

Explore the solution architecture

When you work on smaller projects with smaller teams, it may make sense to have one Azure Machine Learning workspace. The one workspace can be used for everything: to train, test, and deploy your model. However, at Proseware, you want to have a robust and future-proof solution that can easily scale when you build and maintain multiple models that you want to integrate with our web application for practitioners.

To quickly but safely move a model from development to production, you've agreed upon a high-level **machine learning operations** (**MLOps**) architecture.



> **Note**: The diagram is a simplified representation of a MLOps architecture. To view a more detailed architecture, explore the various use cases in the MLOps (v2) solution accelerator.

The architecture includes:

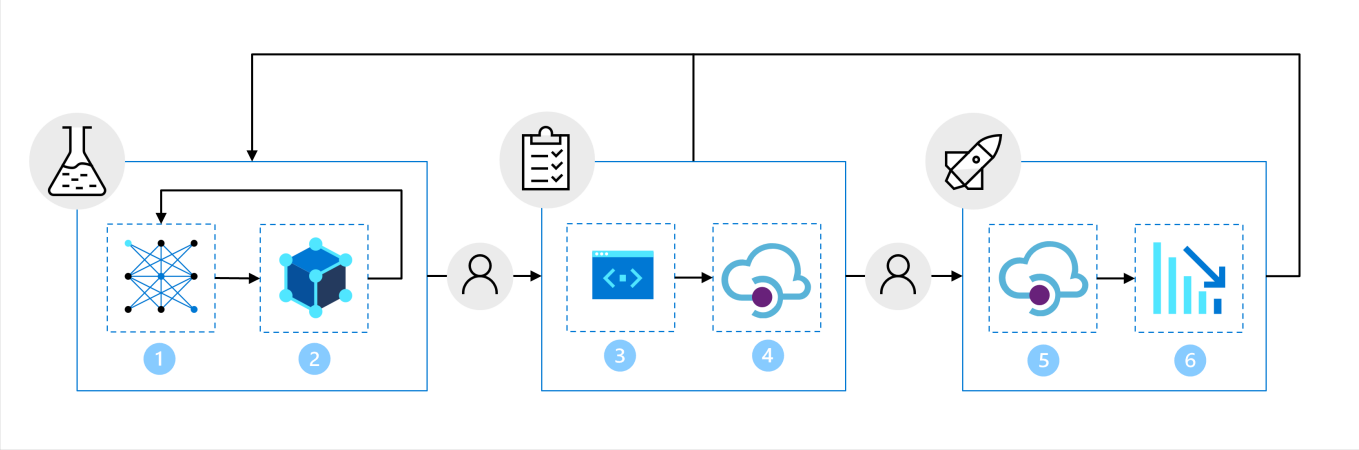
1. **Setup**: Create all necessary Azure resources for the solution.
2. **Model development (inner loop)**: Explore and process the data to train and evaluate the model.
3. **Continuous integration**: Package and register the model.
4. **Model deployment (outer loop)**: Deploy the model.
5. **Continuous deployment**: Test the model and promote to production environment.
6. **Monitoring**: Monitor model and endpoint performance.

To work with machine learning models at a large scale, Proseware wants to use separate environments for different stages. Having separate environments will make it easier to control access to resources. Each

environment can then be associated with a separate Azure Machine Learning workspace.

> **Note**: In this module, we refer to the DevOps interpretation of environments. Note that Azure Machine Learning also uses the term environments to describe a collection of Python packages needed to run a script. These two concepts of environments are independent from each other. Learn more about Azure Machine Learning environments.

To allow for models to be tested before being deployed, you want to work with three environments:



The **development** environment is used for the inner loop:

1. Data scientists train the model.
2. The model is packaged and registered.

The **staging** environment is used for part of the outer loop:

3. Test the code and model with linting and unit testing.
4. Deploy the model to test the endpoint.

The **production** environment is used for another part of the outer loop:

5. Deploy the model to the production endpoint. The production endpoint is integrated with the web application.
6. Monitor the model and endpoint performance to trigger retraining when necessary.

Although many machine learning tasks can and should be automated, you'll also want to plan for points where you want gated approval. When a model has been trained and packaged, you want to notify the lead data scientist to validate the model before it moves to the staging environment.

Similarly, after the model has been tested vigorously in the staging environment, you want to add gated approval to ensure someone from the software development team verifies that all tests were successful before deploying your model to production.

When you work with environments, gated approval allows you to control deployments from one environment to the next.

Set up environments

To implement environments when working with machine learning models, you can use a platform like GitHub. To automate tasks that need to run in separate environments, you'll need to:

- Set up the environments in GitHub.
- Use the environments in GitHub Actions.
- Add approvals to assign required reviewers.

Set up environments in GitHub

To create an environment within your GitHub repo:

1. Go to the **Settings** tab within your repo.
2. Select **Environments**.
3. Create a **new environment**.
4. Enter a name.
5. Select **Configure environment**.

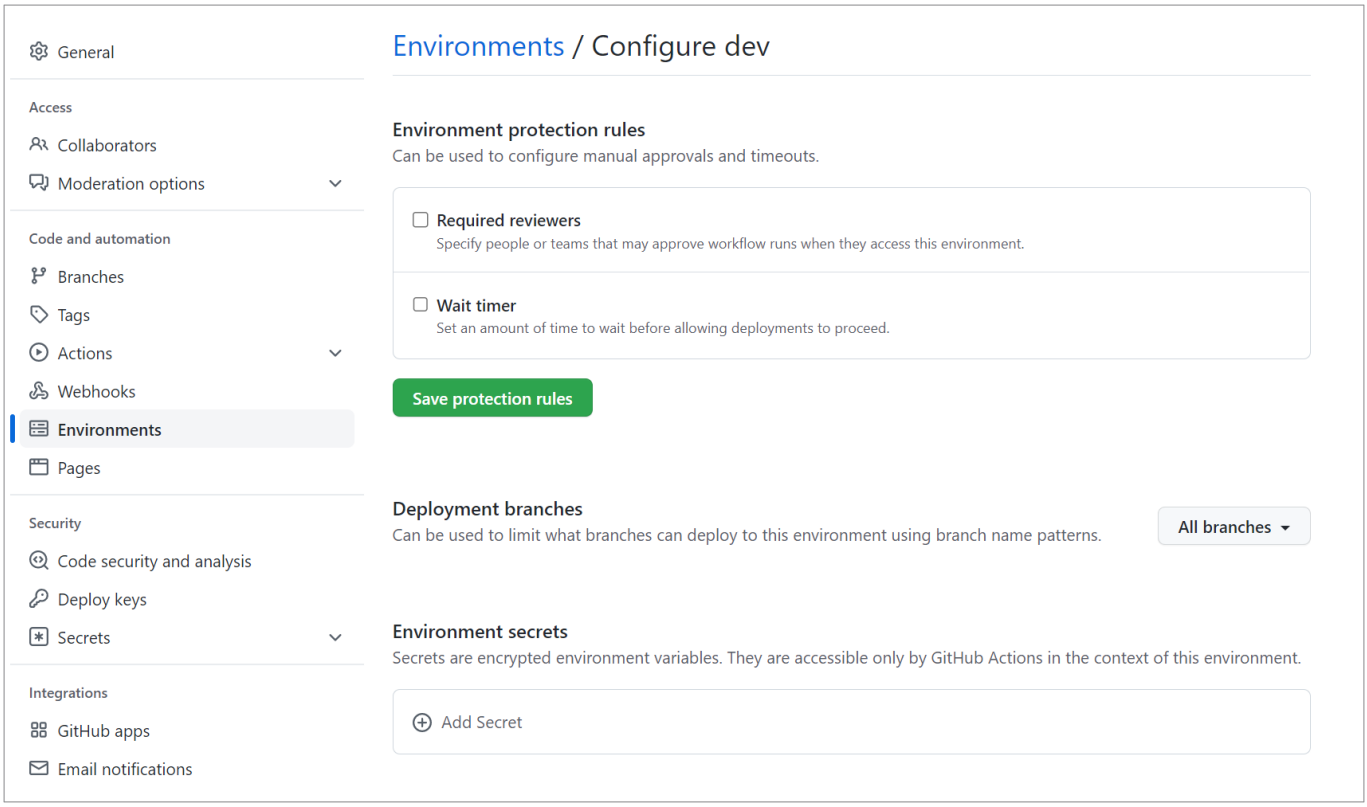
To associate an environment with a specific Azure Machine Learning workspace, you can create an **environment secret** to give only that environment access to an Azure Machine Learning workspace.

> **Note**: To give GitHub access to any Azure Machine Learning workspace, you need to create a service principal in Azure. Next, you need to give the service principal access to the Azure Machine Learning workspace in Azure. Learn how to integrate Azure Machine Learning with DevOps tools such as GitHub.

You can create a secret in the repo to store the credentials of the service principal. When working with environments, you'll want to create an environment secret instead, to define which specific GitHub environment should have access to which Azure Machine Learning workspace.

To create an environment secret, go to the **Environments** tab in the **Settings** tab.

1. Go to your new environment.
2. Navigate to the **Environment secrets** section.



3. Add a new secret.
4. Enter `AZURE_CREDENTIALS` as the name.
5. Enter the service principal credentials in the value field.

Use environments in GitHub Actions and add approvals

After creating environments in your GitHub repo, you can refer to the environment from your GitHub Actions workflows. Whenever you want to add a manual check between environments, you can add **approvals**.

For example, whenever you trigger an Azure Machine Learning job in your GitHub Actions workflow, the task may be executed successfully in the workflow. However, it may be that during model training in the Azure Machine Learning workspace, there's a failure because of an issue with the training script. Or after model training, when you evaluate the model's metrics, you may decide that you need to retrain the model instead of deploying the model.

To give you the opportunity to review the output of the model training in the Azure Machine Learning workspace, you can add an approval for an environment. Whenever a GitHub Actions workflow wants to run a task in a specific environment, the required reviewers will be notified and need to approve the tasks before they'll be run.

> **Tip**: Learn more about how to use environments in GitHub Actions and how to add approvals.

Deploy a model with GitHub Actions

Introduction

You can use the Azure Machine Learning CLI (v2) to deploy trained machine learning models automatically as part of machine learning operations (MLOps).

The data science team you work with has trained a classification model that is able to predict whether someone has diabetes, based on some medical information. Your work as a machine learning engineer, is to establish a process that can automatically deploy the trained model to production.

Using the Azure Machine Learning CLI (v2), you want to set up an automated workflow that will be triggered when a new model is registered. Once the workflow is triggered, the new registered model will be deployed to the production environment.

Understand the business problem

To get value from a machine learning model, you'll have to **deploy** it. Whenever you deploy a model you can generate predictions whenever necessary to give you insights.

At Proseware, a start-up in health care, you've been helping with the development of a web application that will help practitioners diagnose diseases in patients more quickly. When a practitioner enters a patient's medical information, the app will be able to give insights in the probability of that patient having a disease.

The first use case is to help practitioners diagnose diabetes more quickly. After researching medical data, the data science team has trained a model to diagnose whether a patient is likely to have diabetes. The model is accurate enough for implementation. Now, the challenge is to use the model in the web app to generate predictions.

As the model and the app are designed to help the health care practitioner when needed, *you don't want use the model on all the patients*. Instead, you want to give the practitioner the possibility to enter the patient's data into the web app whenever there's reason to belief that patient may have diabetes. To prevent costly and unnecessary tests, the model's predictions on the probability of a patient having diabetes will serve as a first filter to decide who should get tested and who shouldn't.

In the future, more machine learning models to help with diagnosing diseases will be added to the web app. All in order to help the practitioner make more data-driven decisions on which tests should be run to validate that a patient has an illness.

The purpose of the first project is to ensure that a practitioner can enter an individual's medical information in the app, and get a *direct* prediction on the probability of that patient having diabetes. By receiving a direct prediction, the practitioner can use the web app during a consultation with the patient to quickly reach a decision on next steps.

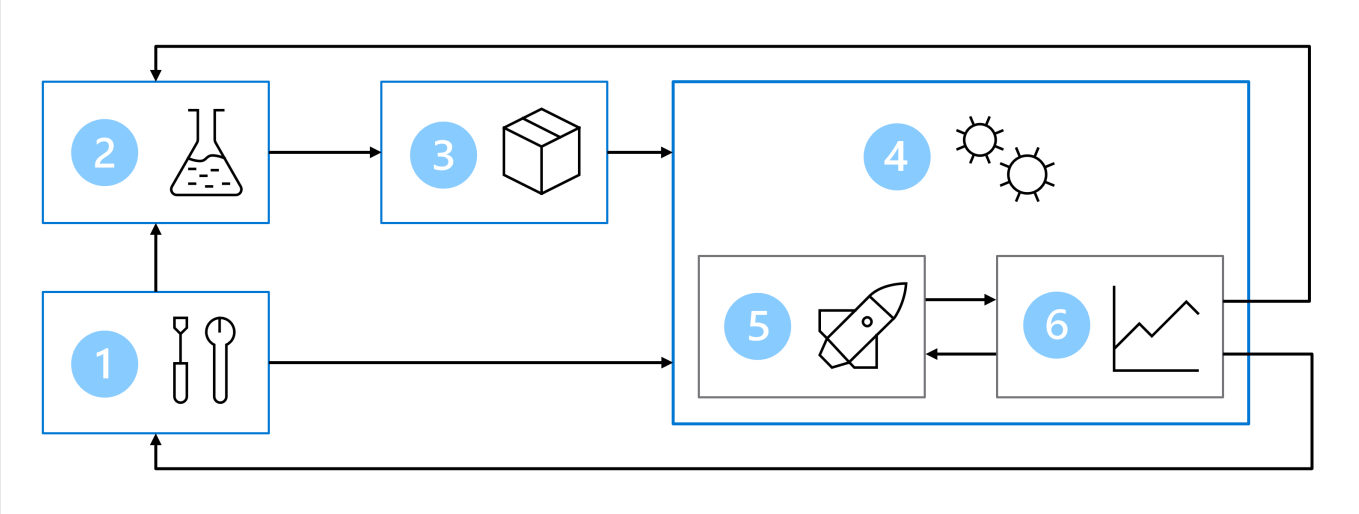
In other words, you need to deploy the model to a **real-time endpoint**. The web app should be able to send the patient's data to the endpoint and get a prediction in return. The prediction should then be visualized in the web app to aid the practitioner.

To deploy a model, you'll want to:

- Register the model.
- Deploy the model.
- Test the deployed model.

Explore the solution architecture

To plan for scale and for automation, you've worked together with several stakeholders to decide on a **machine learning operations** (**MLOps**) architecture.



> **Note**: The diagram is a simplified representation of a MLOps architecture. To view a more detailed architecture, explore the various use cases in the MLOps (v2) solution accelerator.

The architecture includes:

1. **Setup**: Create all necessary Azure resources for the solution.
2. **Model development (inner loop)**: Explore and process the data to train and evaluate the model.

3. **Continuous integration**: Package and register the model.
4. **Model deployment (outer loop)**: Deploy the model.
5. **Continuous deployment**: Test the model and promote to production environment.
6. **Monitoring**: Monitor model and endpoint performance.

Most importantly for the current challenge is to take a model from model development to model deployment. The step in between these two loops is to package and register the model. After the data science team has trained a model, it's essential to package the model, and to register it in the Azure Machine Learning workspace. Once the model is registered, it's time to deploy the model.

There are several approaches to package the model. After reviewing some options like working with pickle files, you've decided with the data science team to work with **MLflow**. When you register the model as an MLflow model, you can opt for no-code deployment in the Azure Machine Learning workspace. when you use no-code deployment, you don't need to create the scoring script and environment for the deployment to work.

When you want to deploy a model, you have a choice between an **online endpoint** for real-time predictions or a **batch endpoint** for batch predictions. As the model will be integrated with a web app where the practitioner will input medical data expecting to get a direct response, you choose to deploy the model to an online endpoint.

You can deploy the model manually in the Azure Machine Learning workspace. However, you expect to deploy more models in the future. And you want to easily redeploy the diabetes classification model whenever the model has been retrained. You therefore want to automate the model deployment wherever possible.

> **Note**: Though automation is a critical aspect of MLOps, it's crucial to maintain a human-in-the-loop. It's a best practice to verify the model before automatically deploying it.

Model deployment

You can manually deploy a model with the Azure Machine Learning workspace. To automatically deploy a model, you can use the Azure Machine Learning CLI (v2) and GitHub Actions. To automatically deploy a model with GitHub Actions, you'll have to:

- Package and register the model.
- Create an endpoint and deploy the model.
- Test the deployed model.

Package and register the model

Whenever you want to deploy a model with the Azure Machine Learning workspace, you'll need to save the model's output and **register** the model in the workspace. When you register the model, you specify whether you have an MLflow or custom model.

When you create and log a model with MLflow, you can use no-code deployment.

> **Tip**: Learn more about how to deploy MLflow models.

To log your model with MLflow, enable autologging in your training script by using `mlflow.autolog()`.

When you log a model during model training, the model is stored in the job output. Alternatively, you can store the model in an Azure Machine Learning datastore.

To register the model, you can point to either a job's output, or to a location in an Azure Machine Learning datastore.

Create an endpoint and deploy the model

To deploy the model to an endpoint, you first create an endpoint and then deploy the model. An endpoint is an HTTPS endpoint that the web app can send data to and get a prediction from. You want the endpoint to remain the same, even after you deploy an updated model to the same endpoint. When the endpoint remains the same, the web app won't need to be updated every time the model is retrained.

> **Tip**: Learn more about how to deploy a model with the Azure Machine Learning CLI (v2).

Test the model

Finally, you'll want to test the deployed model before integrating the endpoint with the web app. Or before converting all traffic of an endpoint to the updated model. You can manually test an online endpoint or you can automate testing the endpoint with GitHub Actions.

> **Note**: You can add a test task to the same workflow as the model deployment task. However, model deployment may take a while to complete. You therefore need to ensure that the testing only happens when the model deployment is completed successfully.